TECHNICAL PORTFOLIO DOSSIER

Implementation of a Zero-Trust SSH Bastion Host with Multi-User Access Control and Network Segmentation

Design, Isolation, and Kernel-Level Auditing of an Enterprise Perimeter Chokepoint

CORE COMPETENCIES SHOWN:

- Network Architecture & Segmentation — Dual-interface design, private subnet isolation
- SSH Hardening & Cryptography — ProxyJump, key-based auth, Zero-Trust enforcement
- Enterprise RBAC & Least Privilege — Multi-team sudo policy, group-based access control
- Stateful Firewalling — nftables, Netfilter, kernel-level packet inspection
- Intrusion Detection & Audit — fail2ban, auditd, privileged command telemetry
- Technical Documentation — 40+ page professional engineering report

ENGINEERING DOMAIN	Networks & Telecommunications Engineering
TARGET ROLE ALIGNMENT	DevSecOps / Cloud Security / Linux Systems Engineering
AUTHOR	Akram Khoulid
PROJECT TYPE	Independent Engineering Lab & Security Case Study

ABSTRACT

In modern enterprise network topologies, the widespread adoption of hybrid engineering workflows and decentralized cloud infrastructure has dramatically expanded the corporate attack surface. Directly exposing administrative entry points, such as standard Secure Shell (SSH) ports on production servers, invites persistent automated brute-force scripts, credential stuffing, and unauthorized network reconnaissance. This technical dossier details the end-to-end engineering, isolation, and cryptographic hardening of a centralized Multi-User SSH Bastion Host designed to mitigate these exact perimeter security vulnerabilities.

Acting as a highly fortified network chokepoint between the untrusted public internet and an isolated internal virtual network, the proposed architecture replaces fragmented authentication boundaries with a single, tightly monitored gateway. The perimeter defenses are reinforced at Layer 4 and Layer 7 using a stateful kernel firewall to implement a default-drop policy, alongside the complete elimination of password-based authentication in favor of high-entropy asymmetric cryptography. Internally, a micro-segmented identity framework separates administrative personnel into role-based groups, enforcing strict discretionary access controls, restricted directory environments, and granular privilege escalation parameters.

To guarantee complete administrative accountability and continuous threat mitigation, the system integrates reactive intrusion prevention engines that dynamically ban hostile external actors trying to brute-force the gateway. Furthermore, low-level kernel auditing telemetry is configured to capture unalterable, forensic transaction logs of all privileged commands executed within the environment. Ultimately, this independent engineering lab demonstrates a highly reproducible, production-grade security baseline aligned with industry compliance standards, serving as a practical showcase of systems administration, defensive network design, and proactive incident auditing.

1 CHAPTER ONE: GENERAL CONTEXT & PROBLEM STATEMENT

1.1 Introduction & Modern Context

In the past, company servers were physical boxes kept inside a locked room in the company office. To manage them, an engineer just had to be physically present in the building. Today, infrastructure has completely changed. Companies run their servers virtually in remote data centers or cloud environments like AWS.

At the same time, engineering teams work remotely from different locations. Because engineers are no longer in the same building as the servers, they must connect to them over the public internet. The standard and most secure protocol used to log into and manage Linux servers remotely is **SSH (Secure Shell)**. While remote access gives engineers the flexibility to maintain systems from anywhere, it also opens up massive security risks if it is not managed correctly

1.2 The Security Risks of Decentralized Access

Imagine a company network with 20 internal servers containing sensitive data, such as web applications, source code, and customer databases. The most common mistake junior administrators make is exposing the default SSH port (Port 22) of every single server directly to the public internet so that engineers can log in from home.

This "decentralized" setup creates three major security nightmares:

- **Too Many Targets (Large Attack Surface):** If all 20 servers are directly connected to the internet, you have 20 public IP addresses exposed. This means a hacker has 20 different doors to try and break through. If just *one* of those servers has an unpatched vulnerability or a weak configuration, the attacker can compromise the entire network.
- **Automated Hacker Bots (Brute-Force Attacks):** The internet is filled with malicious automated software (bots) that scan public IP ranges 24/7 looking for open ports. The moment a bot finds an open Port 22, it immediately launches a brute-force attack—automatically guessing thousands of passwords and SSH keys every single minute until it forces its way in.
- **The Missing Security Camera (Lack of Central Auditing):** When engineers log directly into 20 different servers from 20 different home networks, all the connection

logs are scattered across 20 separate machines. If a database is accidentally or maliciously deleted, the security team has to manually dig through 20 different systems to find out who did it. This makes tracking down incidents an operational nightmare.

1.3 The Solution: The Bastion Host Chokepoint

To solve all three problems at once, this project uses a classic security strategy called a **Chokepoint Design**. Instead of exposing all our servers to the internet, we build an invisible wall around them. We permanently block all direct internet access to our internal application and database servers by placing them inside an isolated **Private Internal Network**.

Then, we open exactly **one heavily guarded front door**: the **Bastion Host** (also known as a Jump Server).

The Bastion Host is a dedicated Linux virtual machine that sits right between the internet and our private network. It becomes the only machine visible to the outside world.

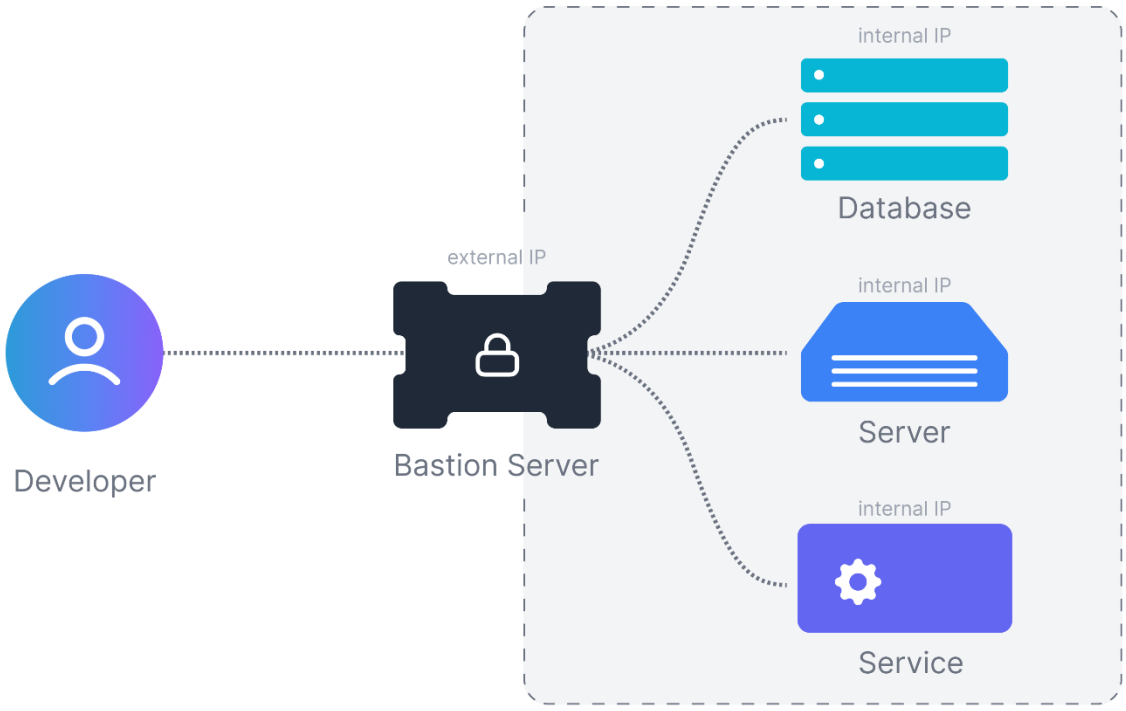


Figure 1: Conceptual Model of a Centralized Perimeter Security Bastion Gateway.

By forcing every single engineer to pass through this single chokepoint before they can "jump" to the internal servers, we gain total control:

1. We reduce our external targets from 20 down to just **one single server** to defend.
2. We can enforce ultra-strict security on this one server, like completely banning passwords and using un-guessable cryptographic keys.
3. We can install a continuous recording system directly inside the Bastion's kernel to log every single command typed by every user, creating an unalterable forensic record of exactly who did what.

1.4 System Limitations & Risks:

While a Bastion Host significantly improves perimeter security, it introduces two critical architectural weaknesses that must be acknowledged:

- **The Single Point of Failure (Availability Risk):** Because all administrative traffic is forced through one single gateway, the availability of the entire network depends entirely on this server. If the Bastion Host crashes, suffers a power failure, or undergoes a Denial of Service (DoS) attack, the entire engineering team is instantly locked out of the backend infrastructure. Production work stops until the Bastion is recovered.
- **The High-Value Target (Security Risk):** By centralizing all access control onto one machine, that machine becomes the ultimate target for hackers. If an attacker successfully compromises the Bastion Host, the defensive wall is broken. The attacker can then use the Bastion’s private network interface to launch internal attacks against all backend application and database servers.

2 CHAPTER TWO : STATE OF THE ART & THEORITICAL FOUNDATIONS

In the previous chapter, we established that directly exposing internal servers to the public internet creates a massive security vulnerability. The architectural solution is to route all remote engineering traffic through a single, heavily fortified chokepoint known as a Bastion Host.

However, a Bastion Host is not a single piece of software that can simply be installed out of the box. Instead, it is a highly customized defensive fortress built by layering several advanced engineering protocols on top of one another. To design a true enterprise-grade perimeter gateway, we must fundamentally shift how the network verifies trust, encrypts data, and filters traffic.

This chapter outlines the three foundational pillars used to secure this environment:

1. **The Architectural Pillar (Zero Trust):** Abandoning the outdated "castle-and-moat" model to build a system that assumes the network is always hostile.
2. **The Authentication Pillar :** Eliminating the human weakness of passwords by relying on the Secure Shell (SSH) protocol, ProxyJump, SSH agent forwarding.
3. **The Network Pillar (Stateful Packet Inspection):** Upgrading from basic static firewalls to intelligent kernel-level filters that actively remember and track the context of every data packet.

2.1 The Architectural Foundation: Zero Trust Security

To understand why modern enterprises require a Bastion Host, we must first understand how network defense strategies have evolved. The tech industry has fundamentally shifted from a "Perimeter Model" to a "Zero Trust Model."

2.1.1 The Flaw of the "Castle-and-Moat" (Perimeter Security)

Historically, corporate networks were designed like medieval castles. The IT team built a massive, heavily defended wall around the outside of the network using a firewall. This was the "moat."

The underlying rule of this old model was **Implicit Trust**:

- If you were outside the wall (on the internet), you were considered dangerous and blocked.
- If you were inside the wall (plugged into an office ethernet cable, or connected via a basic VPN), you were considered safe. Once you passed the front gate, you were implicitly trusted and could freely walk around the internal network, pinging and accessing almost any server.

This model failed catastrophically in the modern era. If a hacker managed to steal a single employee's VPN password, they could bypass the outer wall. Because the internal network automatically trusted anyone inside, the hacker could move laterally from a low-level workstation directly to the core production databases without ever being challenged again.

2.1.2 The Paradigm Shift: Zero Trust Architecture (ZTA)

Zero Trust Architecture was created to solve this exact vulnerability. The core philosophy of Zero Trust is simple: **"Never trust, always verify."** In a Zero Trust network, the location of the user no longer matters. It does not matter if a user is connecting from a coffee shop or sitting at a desk inside the company's secure headquarters. The network assumes that the environment is already compromised.

2.1.3 The Three Core Pillars of Zero Trust

To implement this philosophy technically, a Zero Trust network enforces three strict rules:

1. **Explicit Verification (Identity over Perimeter):** Access is never granted just because a connection comes from a "trusted" IP address. Every single connection request must explicitly prove the user's identity using cryptography before a single byte of data is transferred.
2. **Least Privilege Access:** Even when a user successfully proves their identity, they are not granted access to the whole network. They are given the absolute minimum level of access necessary to do their specific job, for the shortest amount of time possible.
3. **Assume Breach & The "Blast Radius":** Network architects must build the system assuming that a hacker *will* eventually get inside. To survive this, engineers focus on limiting the "Blast Radius" (the scope of impact or disruption that a single failure, misconfiguration, or security incident can cause across connected systems and services).
    - Think of a military submarine. A submarine is divided into multiple watertight compartments. If a torpedo breaches the hull and water floods into one room,

the crew seals the heavy steel doors. The submarine survives because the flood (the blast radius) is contained to a single room.

- In networking, this is called **Micro-segmentation**. We isolate servers into tiny, separate network zones using internal firewalls. If a hacker compromises a web server, the internal blast doors stay shut, preventing them from reaching the database.

2.1.4 How the Bastion Host Enforces Zero Trust in this Project

A Bastion Host is the ultimate physical manifestation of the Zero Trust philosophy. It does not just act as a door; it acts as an active, microscopic security checkpoint that forces every connection to follow the three Zero Trust rules:

- **Enforcing Explicit Verification:** The Bastion is configured to completely reject password authentication. It enforces identity verification by demanding a cryptographic SSH key signature. If a user cannot produce the correct cryptographic math, the Bastion instantly drops the connection at the door.
- **Enforcing Least Privilege:** Once inside the Bastion, users are restricted by strict Linux Role-Based Access Control (RBAC). A junior developer logging into the Bastion is placed in a restricted user group with limited commands, preventing them from modifying system configurations or viewing other users' activities.
- **Enforcing "Assume Breach" (Empty Pockets):** Because we assume the Bastion itself is a massive target for hackers, we configure it to hold absolutely zero credentials for the internal network. By utilizing SSH ProxyJump, the Bastion acts as a blind tunnel. Even if a hacker successfully compromises the Bastion and gains full root control, their blast radius is contained. They find no private keys on the hard drive and cannot move laterally into the internal production servers.

2.2 The Authentication Pillar (Asymmetric Cryptography)

2.2.1 Key-based Authentification:

Instead of typing a password, the server verifies your identity using math. You generate a key pair — a private key that stays on your laptop, a public key that goes on the server. When you connect, the server challenges you with a cryptographic puzzle that only your private key can solve. You never send the password over the network because there is no password.

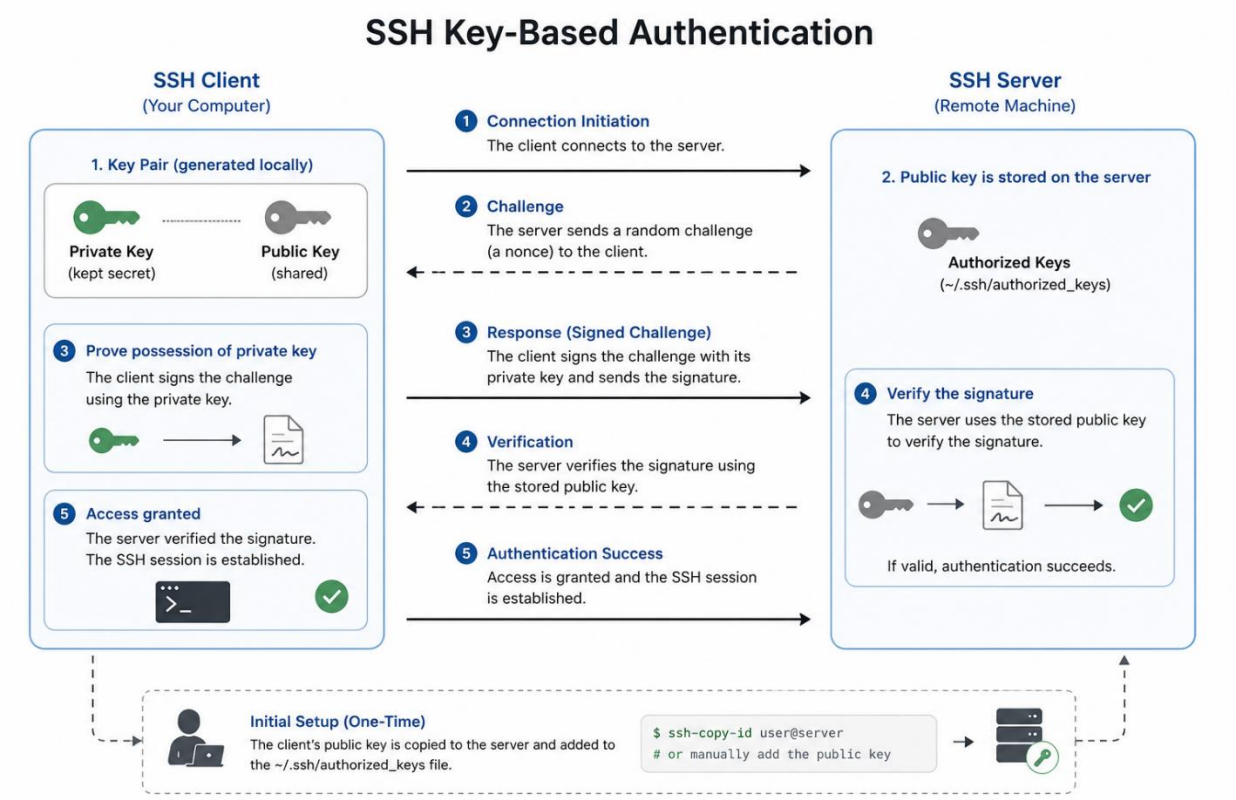


Figure 2: SSH Key-Based Authentification

2.2.2 ProxyJump: The Bastion as a Blind Tunnel

In a traditional SSH setup, reaching an internal server requires two separate, manual steps: the engineer first opens an SSH session to the bastion host, and then opens a second SSH session from inside the bastion to the target server. This approach introduces a critical security flaw — for the second connection to work, the engineer's credentials for the internal servers must either be typed manually each time or, worse, stored as private key files directly on the bastion host's disk. If an attacker ever compromises the bastion, those credentials are immediately exposed, and the internal network falls with it.

ProxyJump was designed to eliminate this flaw entirely. It allows an engineer to establish a single SSH command from their laptop that passes transparently through the bastion and lands directly on the target server, without the bastion ever participating in the authentication process. From the bastion's perspective, it is handling nothing more than a raw stream of encrypted bytes — it cannot read the contents, it cannot extract credentials, and it stores nothing. It is a blind tunnel.

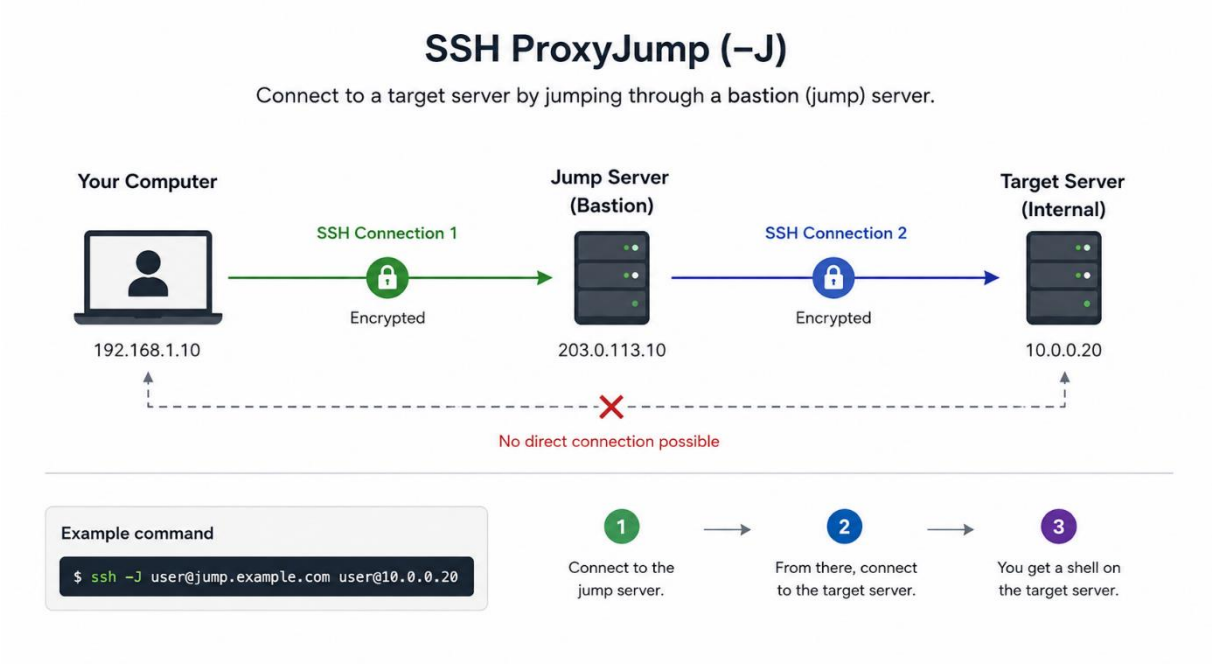


Figure 3: SSH ProxyJump Connection Flow

The practical consequence of this design is significant: even if an attacker gains full administrative control over the bastion host, they find it empty. There are no private keys on the disk, no passwords in memory, and no credentials to steal. The blast radius of a compromised bastion is contained to the bastion itself. The internal servers remain unreachable.

In this project, ProxyJump is the mechanism that makes the architecture in Chapter 3 possible. The developer's laptop is the only machine that holds credentials. The bastion host is the checkpoint that verifies identity. The internal servers are the protected destination. These three roles are strictly separated and never overlap.

2.3 The Network Pillar (Stateful Packet Inspection)

With identity now proven through cryptography and access routed through a blind tunnel, the connection has passed the authentication layer. But a secure system cannot rely on identity verification alone — it must also control, at the network level, which packets are even allowed to reach the bastion in the first place. This is the role of the third and final pillar: Stateful Packet Inspection.

2.3.1 The Limits of Static Firewalls

The earliest generation of network firewalls operated on a simple principle: examine each packet in complete isolation and apply a fixed rule based solely on its source IP address,

destination IP address, and port number. If the packet matched an allowed rule, it passed. If it did not, it was blocked. These are known as static firewalls, or packet-filter firewalls.

This approach has a fundamental weakness. Because each packet is evaluated with no memory of what came before it, a static firewall cannot distinguish between a legitimate packet that belongs to an established connection and a malicious packet that was crafted by an attacker to look like one. An attacker who knows the firewall allows traffic on port 22 can send forged packets on that port at any time, and the static firewall has no mechanism to question whether a real session exists behind them.

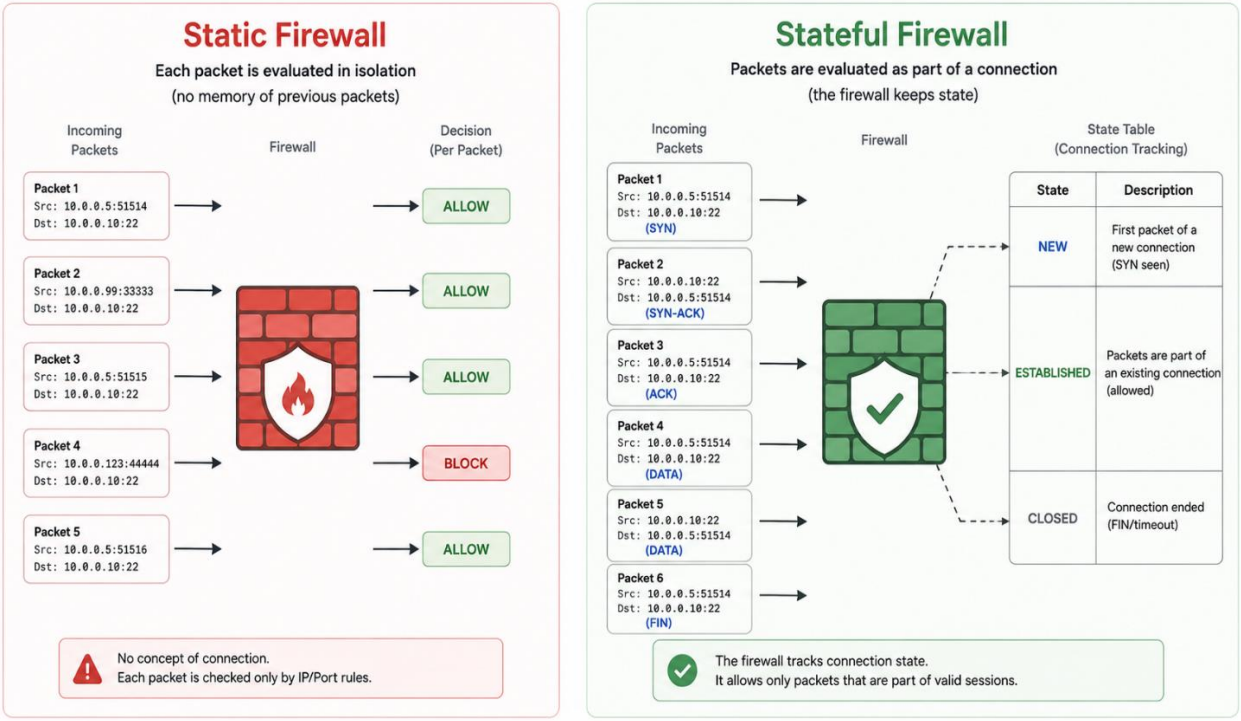


Figure 4: Static firealls vs Stateful firewalls

2.3.2 Stateful Packet Inspection: Firewall with Memory

Stateful Packet Inspection (SPI) was developed to address this exact limitation. A stateful firewall does not evaluate packets in isolation — it tracks the full lifecycle of every network connection passing through it in a structure called the **state table**.

Every connection that passes through the firewall is assigned a state:

State	Meaning
NEW	The first packet of a connection not yet seen before
ESTABLISHED	A packet belonging to a connection already approved and active

RELATED	A packet related to an existing connection (such as an error response)
INVALID	A packet that fits no known connection — dropped immediately

When a new SSH connection arrives on the bastion, the firewall sees the first packet, classifies it as NEW, evaluates it against the rules, and if approved, creates an entry in the state table. Every subsequent packet belonging to that session is classified as ESTABLISHED and passes automatically. If an attacker sends a forged packet claiming to belong to a session that does not exist in the state table, the firewall classifies it as INVALID and drops it silently — no application on the bastion ever sees it.

This transforms the firewall from a passive gate into an active, intelligent checkpoint that understands the context of every byte passing through it.

2.3.3 The Linux Netfilter Framework

In Linux, stateful packet inspection is not implemented by an external appliance or a separate software application. It is built directly into the kernel through a subsystem called **Netfilter**. This is a critical architectural detail: filtering happens at the lowest possible level of the operating system, before any application — including the SSH daemon — ever processes the connection.

Netfilter defines a series of hook points inside the kernel through which every packet must pass. **nftables** is the modern interface used to write rules against these hook points. It replaces the older iptables tool and introduces native stateful tracking, cleaner syntax, and better performance under high traffic loads.

In this project, nftables is the tool used to implement all firewall rules on the bastion host. Its position inside the kernel means that malicious packets are eliminated before consuming any application resources — a connection that fails inspection costs the bastion almost nothing to reject.

The following figure presents the packet-processing pipeline implemented by the Linux kernel. It emphasizes the strategic position of Netfilter, where firewall rules are enforced before packets are delivered to application-level services.

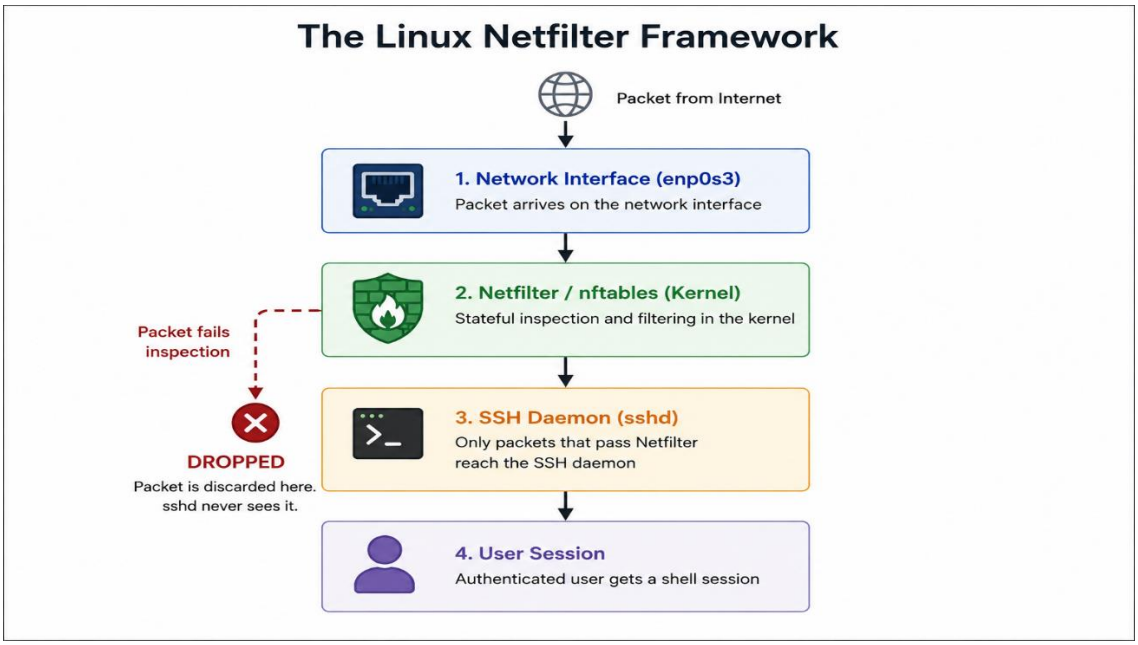


Figure 5: The packet-processing path in linux

3 CHAPTER THIRD: PROJECT ARCHITECHTURE

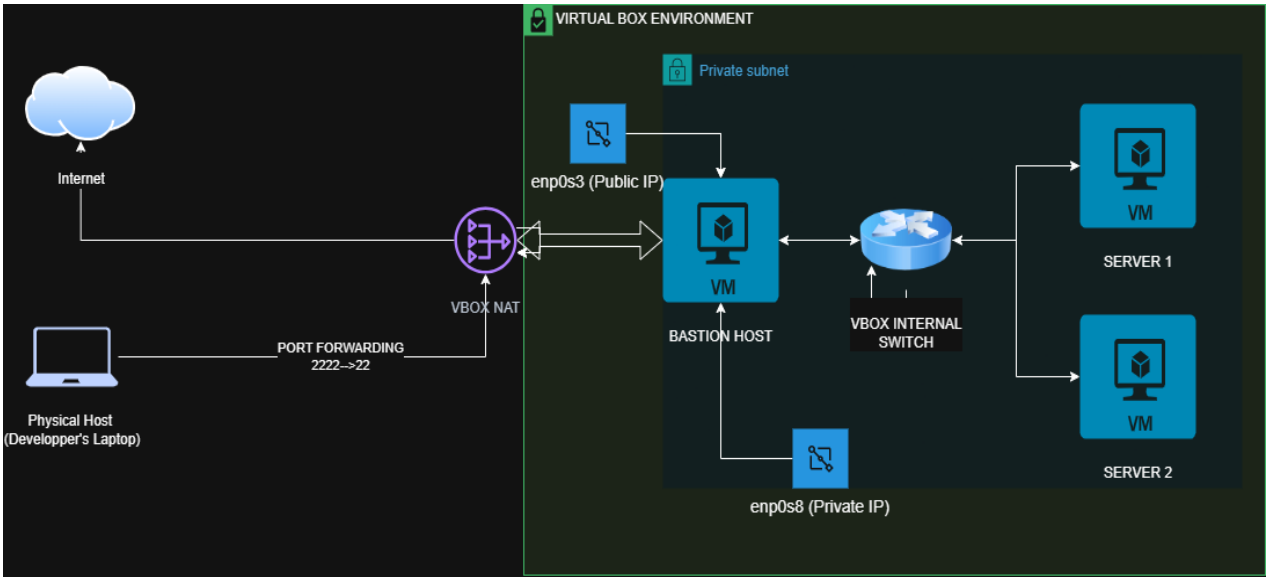
3.1 Architecture Overview

This project simulates an enterprise-grade secure access environment using VirtualBox as the virtualization platform. The infrastructure is composed of three virtual machines connected through two logically separated networks. A single Bastion Host VM sits at the boundary between the external network — reachable from the developer's physical laptop — and a fully isolated private subnet housing two backend servers that are never directly exposed to the outside world.

The Bastion Host is equipped with two network interfaces, each connected to a different network zone. This dual-interface design is the physical enforcement of network segmentation: no direct path exists between the external network and the internal servers except through the bastion, and every connection that passes through it is subject to authentication, firewall inspection, and access control.

3.2 Network Diagram

The diagram below illustrates the complete network topology of the project environment. Each arrow represents the direction of allowed traffic flow. The VirtualBox NAT layer simulates an internet gateway, and port forwarding on port 2222 maps external SSH requests from the physical host to port 22 on the Bastion Host.



3.1.1 Component Summary

Component	Role	Network Zone
Physical Host (Developer's Laptop)	External SSH client — the engineer's workstation	Outside VirtualBox
VirtualBox NAT	Simulates internet gateway — applies port forwarding 2222 → 22	External
Bastion Host VM	Sole entry point to the private subnet — hardened and monitored	Both zones
enp0s3 (Bastion)	Public-facing interface — receives inbound SSH connections	External (NAT)
enp0s8 (Bastion)	Private-facing interface — connects to the internal subnet	Internal
VirtualBox Internal Switch	Simulates a private LAN switch — no external routing	Internal only
Server 1 VM	Backend server — first target reachable via the bastion	Internal only
Server 2 VM	Backend server — second target reachable via the bastion	Internal only

3.2 Network Addressing Plan

Before any configuration begins, a fixed addressing plan is established for all interfaces. Static IP addresses are assigned to every internal interface to ensure that firewall rules, SSH configurations, and ProxyJump directives reference predictable, permanent addresses. The external interface of the Bastion Host receives its address dynamically from the VirtualBox NAT DHCP service.

Interface	Virtual Machine	IP Address	Subnet	Role
enp0s3	Bastion Host	10.0.2.15 (DHCP)	10.0.2.0/24	Public — receives external SSH
enp0s8	Bastion Host	192.168.100.1	192.168.100.0/24	Private — gateway to internal subnet
eth0	Server 1	192.168.100.10	192.168.100.0/24	Internal — backend server 1
eth0	Server 2	192.168.100.20	192.168.100.0/24	Internal — backend server 2

3.3 Traffic Flow Description

The following describes the complete journey of a connection from the moment an engineer opens a terminal on their laptop to the moment they land on an internal server. Understanding this flow is essential — each step corresponds to a security control that will be implemented in the chapters that follow.

Step	Event	Security Control Applied
1	Engineer initiates SSH from their physical laptop targeting localhost port 2222	Connection originates from the trusted developer machine
2	VirtualBox NAT receives the connection and applies port forwarding: 2222 → Bastion port 22	Only port 2222 is forwarded — all other ports are silently dropped by NAT
3	nftables on the Bastion inspects the incoming packet — checks state, port, and source	Stateful firewall drops INVALID packets before sshd ever sees them
4	sshd on the Bastion challenges the client for cryptographic key authentication	Password authentication is disabled — only valid key signatures are accepted
5	Linux RBAC checks whether the authenticated user belongs to an allowed group	Users not in devs, ops, or auditors are rejected regardless of key validity
6	ProxyJump forwards the tunnel through enp0s8 toward the target server	The Bastion acts as a blind pipe — it holds no credentials for internal servers
7	The internal server authenticates the engineer's key via SSH Agent Forwarding	The private key never leaves the laptop — the Bastion never sees it
8	Engineer lands on Server 1 or Server 2 — session is logged by auditd	Full audit trail of every command is recorded on the Bastion

3.4 User and Access Model

Access to the infrastructure is organized around three distinct engineering teams, each mapped to a Linux group. This structure directly implements the Least Privilege principle established in Chapter 2: no user is granted more access than their role strictly requires. The table below defines the access matrix that will be enforced during implementation.

Team	Linux Group	SSH to Bastion	Can Reach	sudo Rights
Development	devs	Yes — key required	Server 1, Server 2	Restart application services only
Operations	ops	Yes — key required	Server 1, Server 2	Full sudo — unrestricted
Audit	auditors	Yes — key required	Server 1 only	Read system logs only — no write access

Users who do not belong to any of these three groups cannot establish an SSH session to the Bastion Host, regardless of whether they possess a valid SSH key. Group membership is the second gate after cryptographic identity.

3.5 Implementation Phases

The project is executed across six sequential phases. Each phase builds on the previous one — it is not possible to harden SSH before the server exists, and it is not possible to write firewall rules before the network interfaces are configured. The roadmap below defines the order of work and the primary deliverable of each phase.

Phase	Title	Primary Deliverable
1	Server Setup and Network Baseline	Three VMs running, static IPs assigned, basic SSH connectivity confirmed
2	Multi-Team User and Group Architecture	Three groups created, home directories set, permissions and umask configured
3	SSH Hardening	sshd_config hardened — root login disabled, password auth off, group restrictions active
4	Firewall Rules with nftables	Stateful ruleset active — only port 22 allowed in, all other traffic dropped
5	Intrusion Detection and Logging	fail2ban blocking brute-force, auditd logging all privileged commands
6	Sudo Policy and Least Privilege	sudoers drop-in files enforcing per-team access — tested and validated

3.6 Tools and Technologies

The table below lists every tool used in this project, the version deployed, and its specific role. All tools are open-source and representative of the standard enterprise Linux stack.

Tool / Technology	Version	Role in This Project
VirtualBox	7.x	Virtualization platform hosting all three VMs
Ubuntu Server	22.04 LTS	Operating system for the Bastion Host and both backend servers
OpenSSH Server	9.x	SSH daemon on the Bastion — configured and hardened in Phase 3
nftables	Current	Kernel-level stateful firewall — ruleset implemented in Phase 4
fail2ban	Current	Brute-force protection — bans IPs after repeated failed auth attempts
auditd	Current	Kernel audit daemon — logs all privileged file access and commands
journalctl	systemd	Primary tool for reading system and SSH service logs
SSH ProxyJump	OpenSSH built-in	Transparent tunnel from laptop to internal servers through the Bastion

3.7 Environment Constraints and Design Decisions

This project runs entirely on a single physical machine inside VirtualBox. This is a deliberate constraint, not a limitation — it makes the environment reproducible on any developer's laptop without requiring cloud accounts, external servers, or network hardware.

The architectural decisions, configuration files, firewall rules, and access control policies produced in this project are identical in structure and intent to those deployed in production enterprise environments. The only element that differs is the underlying hardware. A sysadmin who completes this project can apply every concept directly to a cloud VM, a bare-metal server, or a corporate data center without modification.

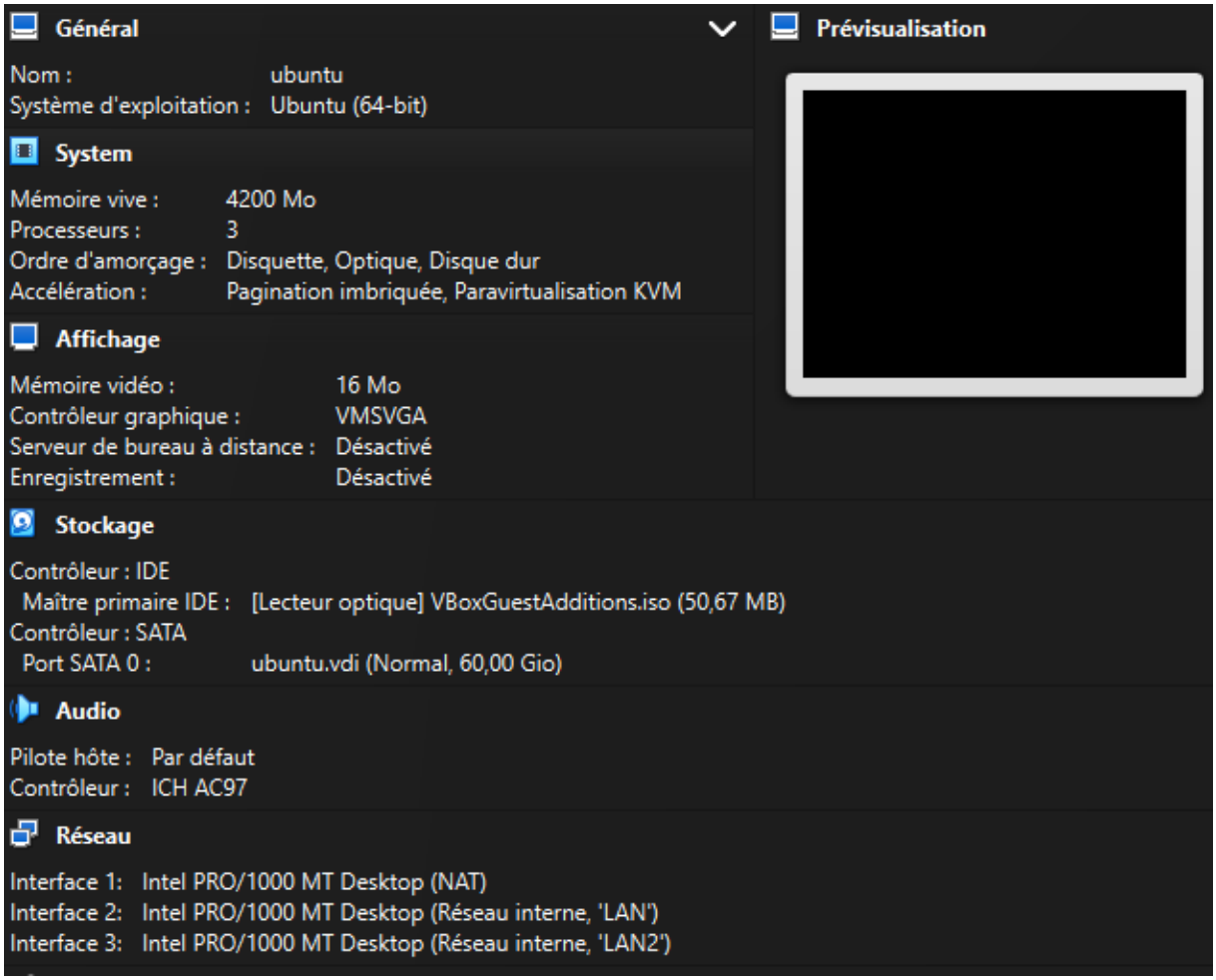
One specific constraint worth noting: because VirtualBox NAT does not assign a public routable IP, port forwarding is used to simulate external access. In a real deployment, the Bastion Host's public interface would hold a static public IP assigned by the cloud provider or network team. The security model is identical — only the addressing mechanism differs.

4 Chapter Four – Bastion Host Implementation

4.1 Environment Preparation

4.1.1 Virtual Machine Creation

The bastion host was deployed as an Ubuntu Server virtual machine using Oracle VirtualBox. Virtualization provides an isolated environment where the server can be configured, tested, and secured without affecting the host operating system. This approach also allows the environment to be recreated easily if required.



The virtual machine was configured with the hardware resources necessary for the project, including CPU, memory, storage, and network adapters.

4.1.2 Network Configuration

The virtual machine was connected to the network using VirtualBox network adapters. The primary adapter was configured in NAT mode to provide Internet access for installing software packages and downloading system updates.

Additional virtual interfaces were available for internal laboratory experiments; however, only the interface used for SSH communication was required for the bastion host implementation.

```
root@akhoulid-VirtualBox:~/netplan-auto# ip link
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen
1000
    link/ether 08:00:27:06:de:3d brd ff:ff:ff:ff:ff:ff
    altname enx08002706de3d
3: ovs-system: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
    link/ether a2:65:4c:e1:8f:26 brd ff:ff:ff:ff:ff:ff
4: vlan20: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN mode DEFAULT group default
len 1000
    link/ether fe:ac:b9:98:b3:3a brd ff:ff:ff:ff:ff:ff
5: bridge12: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN mode DEFAULT group defaul
 qlen 1000
    link/ether 08:00:27:21:45:96 brd ff:ff:ff:ff:ff:ff
6: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master ovs-system state UP mode DEFAULT
roup default qlen 1000
    link/ether 08:00:27:21:45:96 brd ff:ff:ff:ff:ff:ff
    altname enx080027214596
```

4.1.3 Static IP Address Configuration

4.1.3.1 Automated Network Configuration

Instead of configuring the network interfaces manually, an Ansible playbook developed during previous laboratory work was used to automate the network configuration process. Automation ensures that the same configuration can be applied repeatedly with consistent results while reducing the possibility of human error.

The playbook performs several tasks, including applying the Netplan configuration, assigning IP addresses to the required interfaces when necessary, activating network interfaces, restarting the DHCP service, verifying the network configuration, and enabling Internet access through Network Address Translation (NAT). Using Ansible also simplifies future deployments by allowing the complete network configuration to be reproduced with a single command.

```
- name: Check enp0s8 current addresses
  ansible.builtin.command:
    cmd: ip addr show dev enp0s8
  register: enp0s8_ip
  changed_when: false

- name: Bring up enp0s8 and assign IP
  ansible.builtin.command:
    cmd: ip addr add 192.168.10.1/24 dev enp0s8
  when: "'192.168.10.1' not in enp0s8_ip.stdout"

- name: Set enp0s8 up
  ansible.builtin.command:
    cmd: ip link set enp0s8 up
  when: " 'UP' not in enp0s8_ip.stdout"
```

4.1.3.2 Network Interface Verification

After executing the playbook, the network configuration was verified using the ip a command, which displays all network interfaces available on the system together with their operational state and assigned IP addresses.

For the bastion host implementation, the most important interfaces are:

- **enp0s3**, which provides Internet connectivity through the VirtualBox NAT network.
- **bridge12**, which serves the internal laboratory network and uses the static IP address **192.168.10.1/24**.

The remaining interfaces are associated with other laboratory components, such as Open vSwitch and Docker, and are not directly involved in the bastion host deployment

```
    valid_lft forever preferred_lft forever
enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
 link/ether 08:00:27:06:de:3d brd ff:ff:ff:ff:ff:ff
 altname enx08002706de3d
 inet 10.0.2.15/24 metric 100 brd 10.0.2.255 scope global dynamic enp0s3
    valid_lft 86359sec preferred_lft 86359sec
 inet6 fd17:625c:f037:2:a00:27ff:fe06:de3d/64 scope global dynamic mngtmpaddr noprefixroute
    valid_lft 86360sec preferred_lft 14360sec
 inet6 fe80::a00:27ff:fe06:de3d/64 scope link proto kernel_ll
    valid_lft forever preferred_lft forever
ovs-system: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
 link/ether a2:65:4c:e1:8f:26 brd ff:ff:ff:ff:ff:ff
vlan20: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default qlen 1000
 link/ether fe:ac:b9:98:b3:3a brd ff:ff:ff:ff:ff:ff
bridge12: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default qlen 1000
 link/ether 08:00:27:21:45:96 brd ff:ff:ff:ff:ff:ff
enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel master ovs-system state UP group default
en 1000
 link/ether 08:00:27:21:45:96 brd ff:ff:ff:ff:ff:ff
 altname enx080027214596
 inet 192.168.10.1/24 scope global enp0s8
    valid_lft forever preferred_lft forever
 inet6 fe80::a00:27ff:fe21:4596/64 scope link proto kernel_ll
    valid_lft forever preferred_lft forever
```

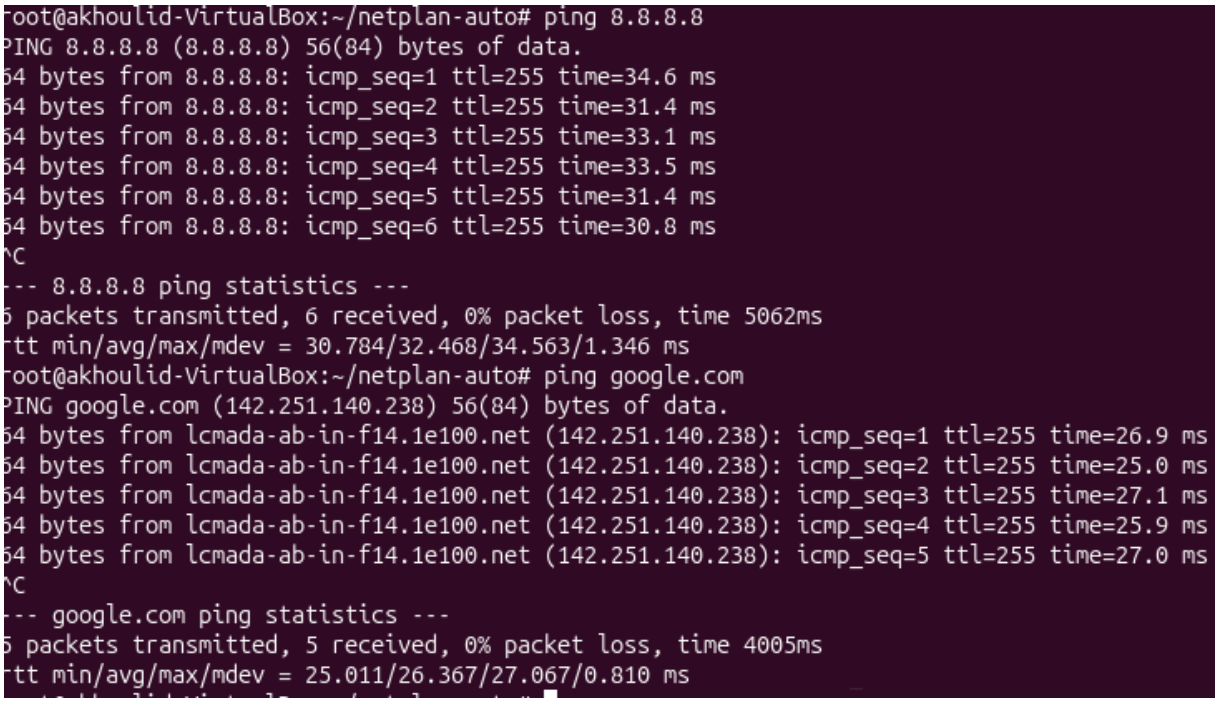
4.1.4 Connectivity Verification

Before configuring SSH, basic network connectivity was verified to ensure that the server could communicate correctly with other devices.

Connectivity tests confirmed that:

- the network interface was operational;
- the server responded to ICMP echo requests;
- the server had Internet connectivity for package installation.

Verifying connectivity before configuring remote administration helps isolate networking issues from SSH configuration problems.



4.2 Operating System Preparation

4.2.1 Updating the System

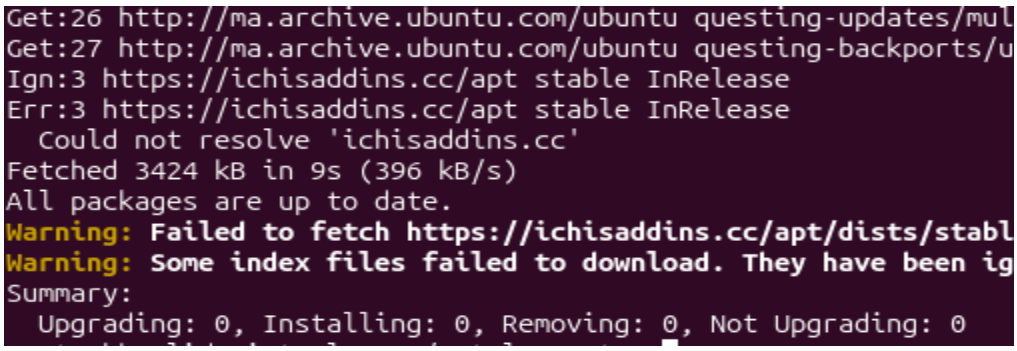
Before deploying the SSH service, the operating system was updated to ensure that the latest security patches and software versions were installed. Keeping the system up to date reduces known vulnerabilities and provides a stable foundation for the bastion host.

The system packages were updated using the following commands:

sudo apt update

sudo apt upgrade

Although no additional packages required upgrading during the final verification, executing these commands remains an essential best practice before deploying any server.



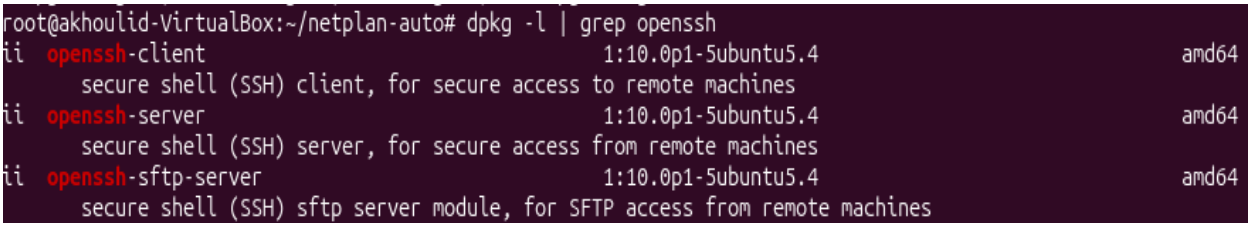
4.2.2 Installing OpenSSH Server

The OpenSSH Server package was installed to provide secure remote administration capabilities. Installing this package adds the SSH daemon (sshd), the systemd service files, the default configuration files, and the cryptographic host keys required to accept secure SSH connections.

The package was installed using:

sudo apt install openssh-server

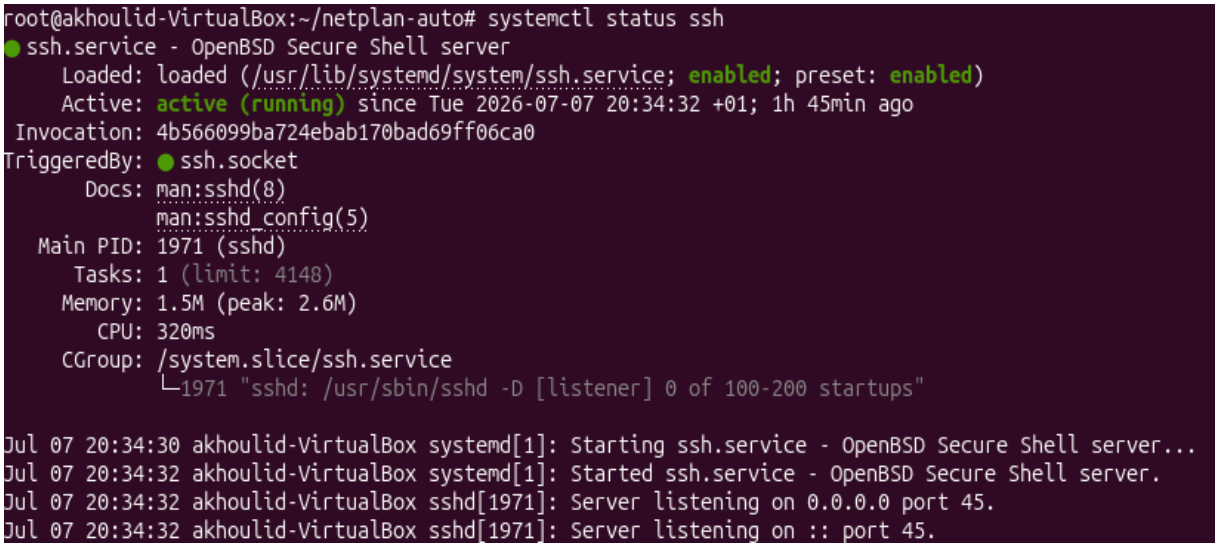
During verification, the package was already present because it had been installed previously during the laboratory setup.



4.2.3 Verifying the SSH Service

After installation, the SSH service was verified to ensure that it was running correctly and ready to accept incoming connections.

The service status was checked using:



Verification confirmed that:

- the service was active,
- the SSH daemon was running,
- the server was listening for incoming SSH connections.

This validation ensures that the SSH server is operational before proceeding with further configuration and hardening.

4.3 SSH Configuration

4.3.1 Exploring sshd_config

The SSH daemon on Linux is controlled by a single configuration file located at /etc/ssh/sshd_config. This file is read by the sshd process at startup and each time the service is explicitly reloaded. Any change made to this file has no effect on the running daemon until a reload is issued — the service continues operating on its previous configuration until instructed otherwise.

The file uses a simple Directive Value syntax. Lines beginning with # are comments and are ignored by the daemon. The default installation ships with most directives commented out, which means the daemon silently falls back to its built-in defaults for anything not explicitly

defined. In a hardened environment, relying on implicit defaults is not acceptable — every security-relevant directive must be explicitly set and documented.

```
#
#Port 22
#AddressFamily any
#ListenAddress 0.0.0.0
#ListenAddress ::

#HostKey /etc/ssh/ssh_host_rsa_key
#HostKey /etc/ssh/ssh_host_ecdsa_key
#HostKey /etc/ssh/ssh_host_ed25519_key

# Ciphers and keying
#RekeyLimit default none

# Logging
#SyslogFacility AUTH
#LogLevel INFO

# Authentication:

LoginGraceTime 120
#PermitRootLogin prohibit-password
#StrictModes yes
MaxAuthTries 10
#MaxSessions 10

#PubkeyAuthentication yes
```

The default file is heavily commented, which makes it difficult to identify what the daemon is actually enforcing at runtime. To extract only the directives that are actively applied — stripping all comments and blank lines — we use the following command:

```
root@akhoulid-VirtualBox:/etc/ssh# grep -v "^#" /etc/ssh/sshd_config | grep -v "^$"
Include /etc/ssh/sshd_config.d/*.config
LoginGraceTime 120
MaxAuthTries 10
KbdInteractiveAuthentication no
UsePAM yes
X11Forwarding yes
PrintMotd no
MaxStartups 100:30:200
AcceptEnv LANG LC_* COLORTERM NO_COLOR
Subsystem       sftp    /usr/lib/openssh/sftp-server
PermitRootLogin yes
root@akhoulid-VirtualBox:/etc/ssh#
```

This output represents the true running posture of the SSH daemon before any hardening is applied. The active directives and their implications are the following:

Directive	Default Value	What it means
Include	/etc/ssh/sshd_config.d/*.conf	Loads additional config files from this directory — directives there override this file
LoginGraceTime	120	The daemon waits 120 seconds for a client to authenticate — after that the connection is dropped
MaxAuthTries	10	Allows 10 authentication attempts per connection before disconnecting — far too permissive
KbdInteractiveAuthentication	no	Keyboard-interactive auth disabled — only password and key methods apply
UsePAM	yes	Pluggable Authentication Modules active — handles session setup and account checks
X11Forwarding	yes	Allows graphical application forwarding — unnecessary on a bastion and increases attack surface
PrintMotd	no	Suppresses the message of the day on login
MaxStartups	100:30:200	Allows up to 200 simultaneous unauthenticated connections — dangerously high for a bastion
AcceptEnv	LANG LC_* COLORTERM NO_COLOR	Accepts these environment variables from the client — minor risk, acceptable
Subsystem sftp	/usr/lib/openssh/sftp-server	SFTP enabled — not needed on a bastion, should be disabled
PermitRootLogin	yes	Root login fully enabled with no restriction — a critical security flaw

The default configuration is functional but entirely unsuitable for a production-hardened environment. Password authentication is enabled, root login is only partially restricted, and no access control exists at the user or group level. The sections that follow address each of these weaknesses systematically, starting with the daemon's listening port.

4.3.2 Configuring the SSH Daemon

With a clear picture of the default state, we now apply the baseline configuration directives. These changes define how the daemon operates — the port it listens on, the interfaces it accepts connections from, session behavior, and connection limits. Security-critical directives such as root login and password authentication are addressed separately in section 4.3, keeping a clean separation between operational configuration and hardening decisions.

```
root@akhoulid-VirtualBox:/etc/ssh# grep -v "^#" /etc/ssh/sshd_config | grep -v "^$"
Include /etc/ssh/sshd_config.d/*.config
Port 22
ListenAddress 10.0.2.15
LoginGraceTime 30
MaxAuthTries 3
PubkeyAuthentication yes
PasswordAuthentication no
PermitEmptyPasswords no
KbdInteractiveAuthentication no
UsePAM yes
X11Forwarding no
PrintMotd no
ClientAliveInterval 300
ClientAliveCountMax 3
MaxStartups 10:30:60
AcceptEnv LANG LC_* COLORTERM NO_COLOR
Subsystem       sftp    /usr/lib/openssh/sftp-server
PermitRootLogin no
```

4.3.2.1 Port 22

Port 22 is the universally recognized standard port for SSH communication. Changing it is sometimes proposed as a security measure, but in practice it offers no real protection — automated scanners probe all 65,535 ports routinely and identify SSH services regardless of which port they run on. More importantly, keeping port 22 ensures compatibility with every SSH client and tool without requiring additional configuration on the connecting side.

In this project there is a second and more decisive reason to keep port 22: the bastion host sits behind VirtualBox NAT and has no real public IP address — unlike a cloud server on AWS or Azure which receives a routable public IP directly on its interface. The only way to reach the bastion from the physical host is through a port forwarding rule defined in VirtualBox, which

maps port 2222 on the physical machine to port 22 on the bastion. Changing the listening port would require updating this forwarding rule in lockstep — an unnecessary complication that adds no security value. The constraint imposed by the NAT environment actually reinforces a good engineering habit: understand why a port is chosen before changing it, rather than changing it for the illusion of security.

4.3.2.2 ListenAddress 10.0.2.15

By default the SSH daemon listens on all available network interfaces simultaneously — including both enp0s3 and enp0s8. This means the bastion would accept incoming SSH connection attempts from the internal subnet, which directly contradicts the architecture defined in Chapter 3. A bastion host must have one face toward the outside world and one face toward the internal network — these two roles must never blur.

Rather than leaving the daemon listening on 0.0.0.0 and relying solely on firewall rules to block internal SSH attempts, we assign a static IP address to enp0s3 and bind the daemon explicitly to that address. This mirrors exactly what happens in a real cloud deployment: on AWS, an engineer assigns an Elastic IP to the public interface of a bastion EC2 instance, and the SSH daemon is bound to that fixed address. The static IP makes the binding permanent and predictable — the daemon will always know which interface to use regardless of network changes. This is a deliberate choice to simulate production conditions rather than take the shortcut that the local environment would allow.

4.3.2.3 LoginGraceTime 30

The default value of 120 seconds gives an unauthenticated connection two full minutes to complete the authentication process. During this window, that connection occupies a slot in the daemon's connection table. An attacker can deliberately open large numbers of connections and hold them in this unauthenticated state, exhausting the bastion's capacity to accept legitimate connections — a low-effort denial of service. Reducing the grace period to 30 seconds is sufficient for any legitimate client with a properly configured SSH key, which authenticates in under one second.

4.3.2.4 MaxAuthTries 3

Ten authentication attempts per connection provides significant room for automated brute-force tools to operate. A legitimate engineer authenticating with an SSH key never requires more than one attempt. Three tries are retained as a grace margin for genuine human error —

a mistyped passphrase, a wrong key selected — before the connection is forcibly closed. This significantly raises the cost of brute-force attacks without inconveniencing legitimate users.

4.3.2.5 MaxStartups 10:30:60

This directive controls the daemon's behavior under high volumes of simultaneous unauthenticated connections. The format start:rate:full means: begin randomly rejecting 30% of new unauthenticated connections once 10 exist simultaneously, and reject all new ones once 60 exist. The default values of 100:30:200 are appropriate for a public-facing server handling many concurrent users. A bastion host serving a small engineering team will never legitimately see more than a handful of simultaneous unauthenticated connections — the reduced values reflect the actual use case and protect against connection-flood attacks.

4.3.2.6 ClientAliveInterval 300 / ClientAliveCountMax 3

These two directives work together to detect and terminate dead or abandoned sessions. Every 300 seconds, the daemon sends a keepalive message to the connected client. If the client fails to respond to 3 consecutive messages — indicating the connection is dead, the laptop closed, or the network dropped — the session is terminated. Without these settings the daemon never actively checks whether a client is still present, leaving potentially stale sessions open indefinitely. On a bastion host where every active session represents an open channel into the infrastructure, abandoned sessions are an unacceptable risk.

4.3.2.7 X11Forwarding no

X11 forwarding enables graphical application windows to be rendered on the client machine through the SSH tunnel. A bastion host runs no graphical applications and serves no use case that requires this feature. Disabling it removes a non-trivial attack surface — X11 forwarding has been the source of multiple exploitable vulnerabilities in OpenSSH's history — at zero operational cost.

4.4 User and Access Management

4.4.1 Defining the Access Policy

Before creating any user or group on the system, the access policy must be defined on paper. This is a fundamental principle of enterprise access management — permissions are never assigned reactively or adjusted by trial and error. Every role, every permission, and every restriction is a deliberate decision made before a single command is executed.

This project simulates an engineering organization with three distinct roles. Each role maps directly to a Linux group. The group is the unit of permission — not the individual user. This means adding a new engineer to the team requires only assigning them to the correct group, and revoking access requires only removing them from it. The permissions themselves never change.

Role	Linux Group	SSH to Bastion	Reaches	sudo Rights	Cannot
Developer	devs	Yes — key only	Server 1, Server 2	Restart app service only	Modify system config, read auth logs, create users
Operations	ops	Yes — key only	Server 1, Server 2	Full sudo	Nothing — but all actions are logged
Auditor	auditors	Yes — key only	Server 1 only	Read logs only	Write anywhere, restart services, reach Server 2

This table is the single source of truth for every permission decision in this project. Every command in the sections that follow is a direct implementation of one cell in this matrix. If a permission is not in this table, it does not exist on the system.

One important note on the Operations role: full sudo access is not a security weakness if it is properly logged and monitored. In enterprises, the ops team must be able to respond to any incident at any time — restricting their access would make the system unmanageable in a crisis. The control mechanism for ops is not permission restriction but complete audit visibility. Every command ops runs is recorded by the audit daemon. The access is wide — the accountability is total.

4.4.2 Creating Groups

The first concrete step in implementing the access policy is creating the three Linux groups that will serve as the foundation of the entire permission model. In Linux, a group is not just a label — it is a security principal that the kernel uses to make access control decisions on every file, directory, and sudo rule in the system. Creating the groups before the users is not optional — a user cannot be assigned to a group that does not exist.

```
root@akhoulid-VirtualBox:/home# groupadd devs && groupadd audit && groupadd ops
```

Then we verify with this command:

```
root@akhoulid-VirtualBox:/home# grep -E "^devs|^ops|^audit" /etc/group
devs:x:1009:
ops:x:1010:
audit:x:1011:
```

Each line in /etc/group follows the format groupname:password:GID:members. The x in the password field indicates that group passwords are managed by the shadow suite — group passwords are rarely used in modern systems. The GID (Group ID) is the number the Linux kernel uses internally to identify the group — when the kernel checks whether a user has permission to access a file, it compares GID numbers, not group names. The members field is currently empty because no users have been assigned to these groups yet — that happens in the next section.

4.4.3 Creating Users

With the groups established, we now create one user per role. Each user account represents an engineer identity on the system. The user is assigned to their group at creation time — this determines their role from the moment the account exists**.**

```
root@akhoulid-VirtualBox:~# useradd -m -s /bin/bash -g devs -c "Developer User" developer1
useradd -m -s /bin/bash -g ops -c "Operations User" ops1
useradd -m -s /bin/bash -g auditors -c "Auditor User" auditor1
```

Every flag in this command is a deliberate decision:

Flag	Value	Why
-m	—	Creates the home directory automatically — without this the user has no personal workspace and SSH may reject the connection
-s	/bin/bash	Assigns bash as the login shell — gives the user a fully functional interactive environment when they connect
-g	group name	Sets the primary group — every file this user creates will be owned by this group by default
-c	description	Documents the account purpose — visible in system logs and user listings, essential for auditability

Now we set password for each user we created

```
root@akhoulid-VirtualBox:~#
passwd developer1
passwd ops1
passwd auditor1
```

Passwords are set here not for SSH authentication — which uses keys exclusively — but for local user switching using the su command during testing and validation. When we verify that developer1 cannot access auditor1's files, we switch to developer1 locally and attempt the access. Without a password, local switching is not possible.

Then verify everything was created correctly

```
root@akhoulid-VirtualBox:~# grep -E "developer1|ops1|auditor1" /etc/passwd
developer1:x:1010:1009:Developer User:/home/developer1:/bin/bash
auditor1:x:1011:1011:Auditor user:/home/auditor1:/bin/bash
ops1:x:1012:1010:operations user:/home/ops1:/bin/bash
```

The /etc/passwd format is **username:x:UID:GID:comment:home:shell.** The x in the password field means the actual password hash is stored in /etc/shadow, readable only by root. The UID is the user's unique numeric identity — the kernel uses this number, not the username, for every permission check. Notice each user's GID matches the GID of their assigned group from the previous section — this is the link between the user and their role.

Then one final check:

```
root@akhoulid-VirtualBox:~# id developer1
id ops1
id auditor1
uid=1010(developer1) gid=1009(devs) groups=1009(devs)
uid=1012(ops1) gid=1010(ops) groups=1010(ops)
uid=1011(auditor1) gid=1011(audit) groups=1011(audit)
root@akhoulid-VirtualBox:~#
```

The id command shows the complete identity of each user as the kernel sees it — their UID, their primary GID, and all supplementary groups. At this point each user belongs to exactly one group, which is exactly what least privilege requires. No user has been granted membership in any group beyond their role.

4.4.4 Home Directory Permissions

Ubuntu creates home directories with mode 755 by default — meaning every user on the system can enter and list the contents of any other user's home directory. On a general-purpose workstation this is acceptable. On a bastion host where home directories contain SSH authorized keys, this is unacceptable. Any user who can read another user's authorized_keys file gains intelligence about who has access to what — information that should be strictly private.

We fix the permissions with these commands:

```
root@akhoulid-VirtualBox:~# chmod 700 /home/developer1
chmod 700 /home/ops1
chmod 700 /home/auditor1
```

Mode 700 means the owner has full access — read, write, and execute — while group members and all other users have zero access. The execute bit on a directory controls the ability to enter it — without execute permission, a user cannot cd into the directory even if they know it exists. This ensures that developer1 cannot enter ops1's home directory, ops1 cannot enter auditor1's, and so on. Each user's workspace is completely private.

```
drwx------  2 auditor1   audit     4096 Jul 11 21:47 auditor1
drwx------  6 developer1 devs      4096 Jul 11 22:30 developer1
drwx------  6 developer2 devs      4096 Jul 11 22:34 developer2
-rw-r--r--  1 root       root       525 Dec  9  2025 firstcode.py
drwxr-xr-x  2 root       root      4096 Dec  9  2025 network-monitor
drwx------  2 ops1       ops       4096 Jul 11 21:48 ops1
```

One deliberate decision worth documenting: some organizations use 750 instead of 700 to allow the ops team to inspect home directories during security incidents. On this bastion host, home directories contain almost nothing — only SSH configuration files. There is no operational content here to inspect. The 700 choice reflects this reality — if ops needs to investigate a user's SSH keys during an incident, they do so with a specific sudo command that is logged by the audit daemon, not through permanent open access.

4.4.5 SSH Key Setup

SSH key-based authentication is the cornerstone of this project's security model. Password authentication has been disabled entirely on the bastion host — the only way to authenticate is by presenting a valid cryptographic key. This section establishes the key infrastructure for each user account: the SSH directory, the authorized keys file, and the strict permissions that SSH requires before it will accept any key as valid.

Each user account requires the following structure inside their home directory:

```
/home/username/
    .ssh/                          ← directory, mode 700
        authorized_keys            ← file, mode 600
```

The .ssh directory stores SSH-related files for this user. The authorized_keys file contains the public keys that are permitted to authenticate as this user. SSH enforces strict permission requirements on both — if the directory is accessible by anyone other than the owner, or if the authorized_keys file is readable by the group or others, SSH silently ignores the keys entirely and the connection fails with a confusing authentication error.

```
root@akhoulid-VirtualBox:/home# # developer1
mkdir -p /home/developer1/.ssh
touch /home/developer1/.ssh/authorized_keys
chmod 700 /home/developer1/.ssh
chmod 600 /home/developer1/.ssh/authorized_keys
chown -R developer1:devs /home/developer1/.ssh

# ops1
mkdir -p /home/ops1/.ssh
touch /home/ops1/.ssh/authorized_keys
chmod 700 /home/ops1/.ssh
chmod 600 /home/ops1/.ssh/authorized_keys
chown -R ops1:ops /home/ops1/.ssh

# auditor1
mkdir -p /home/auditor1/.ssh
touch /home/auditor1/.ssh/authorized_keys
chmod 700 /home/auditor1/.ssh
chmod 600 /home/auditor1/.ssh/authorized_keys
chown -R auditor1:auditors /home/auditor1/.ssh
```

The .ssh directory — mode 700

The .ssh directory is set to 700 — owner has full access, group and others have nothing. This prevents any other user on the system from listing what files exist inside, from reading their contents, or from modifying them. A user who cannot enter the .ssh directory cannot tamper with the authorized keys, cannot inject their own key, and cannot determine which keys are trusted for this account.

The authorized_keys file --- mode 600

The authorized_keys file is set to 600 — owner can read and write, nobody else can do anything. This is the most sensitive file in a user's home directory. It defines who is cryptographically trusted to act as this user. If group or others had read access, any user on the system could copy the public key and gain intelligence about the authentication setup. If they had write access, they could inject their own public key and authenticate as this user without any credentials.

Ownership — user:primarygroup

The entire .ssh directory tree is owned by the user and their primary group. This ensures that even root-created files inside this directory are correctly attributed to the user. The SSH daemon verifies ownership as part of its security checks — a .ssh directory owned by root but sitting in a user's home would be flagged as suspicious and potentially ignored.

Then we verify:

```
root@akhoulid-VirtualBox:/home/auditor1/.ssh# ls -ld && ls -la
drwx------ 2 auditor1 audit 4096 Jul 13 21:13 .
total 8
drwx------ 2 auditor1 audit 4096 Jul 13 21:13 .
drwx------ 3 auditor1 audit 4096 Jul 13 21:13 ..
-rw------- 1 auditor1 audit    0 Jul 13 21:13 authorized_keys
```

The public keys themselves will be placed in these files in a later chapter when end-to-end connectivity is validated. The structure and permissions established here ensure that when the keys are added, SSH will immediately recognize and trust them. Any deviation from these permissions — even a single wrong bit — causes silent authentication failure that is difficult to diagnose without reading the SSH daemon logs carefully.

4.4.6 Completing SSH Access Control

The user accounts and groups are now established on the bastion host. The final step in SSH access control is informing the SSH daemon which groups are permitted to authenticate. Without this directive, any user account on the system with a valid key can establish an SSH session — including service accounts, temporary accounts, or accounts created for other administrative purposes. The AllowGroups directive creates an explicit whitelist at the daemon level, independent of file permissions or sudo rules.

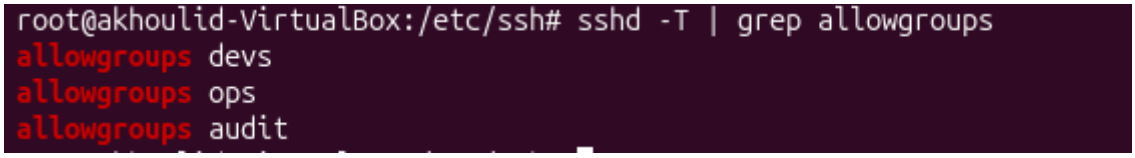
```
#       AllowTcpForwarding no
#       PermitTTY no
#       ForceCommand cvs server
PermitRootLogin no
AllowGroups devs ops audit
```

Then we validate and reload:

```
root@akhoulid-VirtualBox:/# sshd -t
root@akhoulid-VirtualBox:/# systemctl reload sshd
root@akhoulid-VirtualBox:/#
```

The sshd -t command validates the configuration file syntax before the reload is applied — a broken configuration that reaches a live reload would cause the daemon to fail on next restart, potentially locking all engineers out of the bastion. Only after a clean validation is the reload issued. The reload command — as opposed to restart — applies the new configuration without interrupting any sessions that are currently active.

Then we verify it's active



With this directive in place, the SSH access control model on the bastion is complete. Two layers now protect every incoming connection: key-based authentication verifies cryptographic identity, and AllowGroups ensures that only members of the three defined roles can proceed past authentication. A valid key belonging to an account outside these groups is rejected at this final layer. Firewall rules, intrusion detection, and end-to-end ProxyJump validation are addressed in the chapters that follow.

5 Chapter Five — Backend Server Configuration (Kali)

5.1 Introduction

The bastion host configured in Chapter 4 serves a single purpose: verify identity and control who passes through. It holds no sensitive data, runs no applications, and stores no credentials. It is a checkpoint, nothing more.

This chapter deals with an entirely different machine — the backend server running Kali Linux. This is where engineering work actually happens. Code is deployed here, applications run here, logs are written here, and system administration takes place here. The security decisions made in this chapter have direct operational consequences — a misconfigured permission here means a developer cannot deploy their work, or worse, an attacker who obtains a developer's credentials gains access to more than they should.

The three roles defined in the access policy of section 4.2.1 — developers, operations, and auditors — were identities on the bastion. On this server they become real permission boundaries enforced at the filesystem level, the service level, and the privilege level. Each role receives exactly what their job requires and nothing beyond that.

This chapter implements the complete access model on the backend server, starting from SSH configuration through user management, directory permissions, sudo policy, and ending with a full cross-role validation that proves every boundary holds.

5.2 SSH Configuration

The SSH daemon on the backend server is hardened following the same principles applied to the bastion host in Chapter 4, Section 4.1. The complete reasoning behind each directive — disabled root login, key-based authentication only, reduced grace time, connection limits — is documented there and is not repeated here.

One distinction is worth noting. On the bastion, the ListenAddress directive was bound to the public-facing interface enp0s3 at 10.0.2.15 — the interface that receives connections from the outside world. On the backend server there is no public interface. The server has one network interface, eth0, connected exclusively to the internal subnet 192.168.10.0/24. It is physically unreachable from the internet. The ListenAddress is therefore bound to 192.168.10.10 — the only interface that exists, and the only one the bastion can reach.

5.3 User and Group Architecture

5.3.1 Design Principles

The access policy defined in section 4.2.1 established three roles — developers, operations, and auditors — and described what each role is permitted and forbidden to do. On the bastion host, this policy controlled who could pass through. On the backend server, it controls what each engineer can actually do once they arrive.

The implementation follows the same structure as the bastion: groups are created first, users are assigned to groups, and home directories are locked down before any sensitive content is placed inside them. What changes here is the layer beneath — on the bastion, group membership determined SSH access. On this server, group membership determines filesystem access, service control, and sudo privileges. The same identity, a completely different set of consequences.

5.3.2 Creating Groups

The three groups must exist on the backend server independently of the bastion. Linux groups are local to each machine — the bastion's /etc/group and the server's /etc/group are completely separate files with no synchronization between them. Using identical group names across both machines is a deliberate consistency decision: it makes the configuration readable, auditable, and ready for future automation with tools like Ansible that would push identical configurations to every server in the infrastructure.

```
┌──(root㉿kali)-[/home/developer1]
└─# grep -E "^devs|^ops|^auditors" /etc/group
devs:x:1002:
ops:x:1003:
auditors:x:1004:
```

5.3.3 Creating Users

Each user account represents a specific engineering identity. The same usernames are used across the bastion and the backend server — when developer1 passes through the bastion and lands on this server, the system finds a matching account with the correct group membership and home directory waiting. The identity is consistent end to end.

```
┌──(root㉿kali)-[/home/developer1]
└─# grep -E "developer1|developer2|ops1|auditor1" /etc/passwd
developer1:x:1002:1002::/home/developer1:/bin/bash

┌──(root㉿kali)-[/home/developer1]
└─# useradd -m -s /bin/bash -g ops -c "Operations User" ops1

┌──(root㉿kali)-[/home/developer1]
└─# passwd ops1
New password:
Retype new password:
passwd: password updated successfully

┌──(root㉿kali)-[/home/developer1]
└─# useradd -m -s /bin/bash -g auditors -c "Auditor User" audit1

┌──(root㉿kali)-[/home/developer1]
└─# passwd auditor1
passwd: user 'auditor1' does not exist

┌──(root㉿kali)-[/home/developer1]
└─# passwd audit1
New password:
Retype new password:
passwd: password updated successfully
```

Each user belongs to exactly one group at this point. No user has been granted supplementary group memberships beyond their role. This is the starting state — the minimum. Permissions are added from here based on documented operational requirements, never assumed or granted speculatively.

5.3.4 Home Directory Permissions

Home directories on the backend server must follow the same strict permission model applied on the bastion host. Each user's personal workspace must be accessible only to its owner — no other user on the system should be able to enter, list, or read another user's home directory.

```
┌──(root㉿kali)-[/home/developer1]
└─# cd ../ && ls -la
total 28
drwxr-xr-x  7 root       root     4096 Jul 18 08:39 .
drwxr-xr-x 19 root       root     4096 Jul 16 22:39 ..
drwx------ 18 akhoulid   akhoulid 4096 Jul 12 18:24 akhoulid
drwx------  5 audit1     auditors 4096 Jul 18 08:39 audit1
drwx------  6 developer1 devs     4096 Jul 17 20:32 developer1
drwx------ 16 kali       kali     4096 Jan 31 09:27 kali
drwx------  5 ops1       ops      4096 Jul 18 08:38 ops1
```

5.3.5 SSH Key Setup

The SSH directory structure and authorized keys setup follows the same procedure documented in section 4.2.5. The .ssh directory is created for each user with mode 700, the authorized_keys file is created with mode 600, and ownership is assigned to the user and their primary group. The public keys are placed in these files during the end-to-end validation chapter once the complete permission model is in place on both machines.

5.4 Developers Permissions (devs)

5.4.1 Role Definition

A developer's role on the backend server is precise and bounded. Their job is to deploy web application code, monitor application behavior through logs, and restart the web service after deploying changes. Nothing in this role requires access to system configuration, user management, authentication logs, or any service beyond the application they maintain.

This precision is not a restriction imposed on developers out of distrust — it is a protection for them. If a developer's credentials are stolen, the attacker inherits exactly what the developer had: access to the web application directory, the ability to read application logs, and the ability to restart one specific service. The rest of the infrastructure remains untouched. The blast radius is contained to the application layer.

5.4.2 Web Deployment Directory

The web deployment directory is where developers perform their primary function — placing application files that Apache serves to users. By default this directory is owned by root with no group access for developers. The ownership and permission model must be reconfigured to match the access policy

```
┌──(root㉿kali)-[/home]
└─# ls -la /var/www/html && ls -ld /var/www/html
total 24
drwxrws--- 2 root devs  4096 Dec  2  2025 .
drwxr-xr-x 3 root root  4096 Dec  2  2025 ..
-rw-r--r-- 1 root devs 10703 Dec  2  2025 index.html
-rw-r--r-- 1 root devs   615 Dec  2  2025 index.nginx-debian.html
drwxrws--- 2 root devs 4096 Dec  2  2025 /var/www/html
```

The permission 2770 encodes three decisions simultaneously. The 2 sets the SGID bit — every file and subdirectory created inside /var/www/html/ automatically inherits the devs group regardless of which user or process created it. This is essential in a collaborative environment where multiple developers and deployment scripts write files — without SGID, files created by

a deployment script running under a different primary group would be inaccessible to the development team. The 7 for the owner gives root full control for administrative operations. The 7 for the group gives all members of devs full read, write, and execute access — developers can create, edit, and delete files freely. The 0 for others removes all access — no user outside the devs group can enter or list the directory

```
┌──(root㉿kali)-[/home]
└─# cat /etc/profile.d/umask.sh
# set umask 002 for developers
if id -nG "$USER" | grep -qw "devs";
    umask 002
fi
```

Without a umask adjustment, files created by developers default to mode 644 — group members can read but not write. A developer who creates a file blocks their teammates from editing it. The umask 002 changes the default to 664 for files and 775 for directories — group members get full read and write access automatically on every file created. This rule is placed in /etc/profile.d/umask.sh rather than individual .bashrc files because it applies automatically to every current and future member of the devs group without manual configuration per user.

5.4.3 Apache Log Access and Error Logs

Developers need read access to Apache logs and Error Logs to perform basic application debugging. When a deployment breaks the application, the error log contains the exact failure reason. When traffic patterns look suspicious, the access log reveals the source. Without log access, developers cannot diagnose problems independently and must escalate every issue to the operations team — creating a bottleneck that slows down the entire engineering workflow.

The access is intentionally restricted to read-only. Developers must never be able to write to, truncate, or delete log files. Log integrity is a security requirement — logs are evidence. A developer who can delete log entries can erase evidence of their own mistakes or malicious actions.

```
┌──(root㉿kali)-[/home]
└─# ls -la /var/log/apache2/
total 20
drwxr-x---  2 root auditors-devs 4096 Jul 18 08:03 .
drwxr-xr-x 21 root root          4096 Jul 18 20:38 ..
-rw-r-----  1 root auditors-devs    0 Dec  2  2025 access.log
-rw-r-----  1 root auditors-devs 1937 Jul 18 19:18 error.log
-rw-r-----  1 root auditors-devs  746 Jul 18 08:04 error.log.1
-rw-r-----  1 root devs           404 Jul 17 07:52 error.log.2.gz
-rw-r-----  1 root devs             0 Dec  2  2025 other_vhosts_access.log
```

5.4.4 Sudo Policy for Developers

Restarting a system service requires root privileges — only root can send control signals to systemd-managed services. However, granting full sudo access to developers to solve this single operational need would be a severe over-privilege. A developer with unrestricted sudo can restart SSH, modify firewall rules, create users, or read any file on the system. The blast radius of a compromised developer account with full sudo is the entire server.

The solution is a precisely scoped sudo rule that grants exactly one elevated capability and nothing else

```
# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL
%devs   ALL=(root) NOPASSWD: /usr/bin/systemctl restart apache2
```

Every element of this rule is deliberate. %devs applies the rule to every current and future member of the devs group — not to a named individual. ALL= means the rule applies regardless of which terminal or session the command is issued from. (root) means the command executes with root privileges. NOPASSWD: removes the password prompt — developers have no sudo password by design, and since authentication already happened at the SSH key level, demanding a password here adds friction without adding security. /usr/bin/systemctl restart apache2 is the full absolute path of the single permitted command — sudo matches against the exact binary path and arguments, so systemctl restart ssh or any other variant is rejected by a different rule entirely.

5.5 Operations Permissions (ops)

5.5.1 Role Definition

The operations role carries the broadest access on the backend server. Operations engineers are responsible for the complete lifecycle of the infrastructure — installing and configuring services, managing user accounts, responding to security incidents, applying system updates, and maintaining the health of the environment. This scope requires unrestricted administrative access.

In most security frameworks, broad access like this would be considered a risk. In the operations model it is an accepted and managed risk. The control mechanism is not permission restriction — it is complete audit visibility. Every command an operations engineer runs with

elevated privileges is recorded by the system audit daemon with full detail: the user, the time, the exact command, and the arguments. The access is wide. The accountability is total.

Operations engineers are trusted with this access because their role demands it. A security incident at 3am cannot wait for permission escalation. A failed deployment that takes down the application needs immediate administrative intervention. Restricting ops would make the system unmanageable precisely when management matters most

5.5.2 Sudo Policy for Operations

The operations sudo policy is deliberately simple. Complexity in a sudo policy creates gaps — a long list of specific allowed commands will always miss edge cases that ops engineers encounter during incidents. Full sudo with complete audit logging is cleaner, more honest, and more manageable than a complex partial policy that still gets bypassed when an incident demands it.

```
# Allow members of group sudo to execute any command
%sudo   ALL=(ALL:ALL) ALL
%devs   ALL=(root) NOPASSWD: /usr/bin/systemctl restart apache2
%ops    ALL=(ALL:ALL) ALL
```

%ops applies to every member of the ops group. The first ALL means from any host. (ALL:ALL) means ops can execute commands as any user and any group — not just root. The final ALL means any command without restriction. This is the broadest possible sudo grant. It is justified by the operations role and controlled by audit logging configured in Chapter 8.

5.5.3 File System Access

Operations engineers access the filesystem through sudo rather than through direct group permissions. This distinction is important — direct group permissions grant silent, permanent access that is difficult to track. Sudo-mediated access generates an audit log entry for every operation. When ops reads a developer's home directory or a sensitive configuration file, that action is recorded.

5.6 Auditor Permissions (auditors)

5.6.1 Role Definition

The auditor role exists to answer one question independently of the teams being audited: is the system operating according to its documented security policy? Auditors verify that access

controls match documentation, that logs show no unauthorized activity, and that privileged actions are traceable to specific individuals.

The auditor role has one absolute constraint: read-only access everywhere. An auditor who can write to files could tamper with the evidence they are reviewing. An auditor who can restart services could disrupt the system they are auditing. The value of an audit depends entirely on the auditor's inability to alter what they observe. This is enforced at the permission level — not by policy alone, but by the filesystem itself.

In this project, auditors have access to Server 1 only. Their scope covers SSH authentication activity, web application traffic, system events, and privilege usage. They verify the user model, the sudo policy, and the group assignments against the documented access matrix.

5.6.2 Log Access

Auditors access system logs through two mechanisms. Logs including auth.log, syslog, and kern.log are owned by the audit group with mode 640 — readable by group members, invisible to everyone else. auditor1 is a member of the audit group and therefore reads these files directly without any sudo requirement. Apache logs are readable through the shared parent group that gives both developers and auditors access to the application log directory. This design avoids sudo for routine log reading — auditors can grep, tail, and search log files directly without elevated privileges, which is both more efficient and produces less noise in the audit trail.

```
┌──(root㉿kali)-[/home]
└─# ls -la /var/log | grep -E "auth|syslog|kern"
-r--r-----   1 root              audit             17606 Jul 18 23:17 auth.log
-r--r-----   1 root              audit            131566 Jul 18 23:12 kern.log
-r--r-----   1 root              audit            177158 Jul 18 23:17 syslog
```

5.6.3 System Configuration Read Access

Beyond logs, auditors must verify that the running system configuration matches the documented security policy. This requires reading privileged configuration files that are not accessible to regular users. Sudo rules grant auditors read access to exactly these files — no more.

```
%devs   ALL=(root) NOPASSWD: /usr/bin/systemctl restart apache2
%ops    ALL=(ALL:ALL) ALL
%auditors ALL=(root) NOPASSWD: /bin/cat /etc/sudoers
# See sudoers(5) for more information on "@include" directives:
```

5.7 Cross-Role Isolation Validation

5.7.1 Developer isolation test

5.7.2 Auditor isolation test

5.7.3 Blast radius demonstration

The blast radius of a security compromise is defined as the maximum damage an attacker can cause using only the credentials they obtained. This demonstration maps the blast radius of each compromised role to show that the permission model contains breaches rather than allowing them to propagate.

If developer1's credentials are stolen:

The attacker can deploy malicious web content to /var/www/html/ — defacing the site or inserting malicious code. They can read application logs to gather intelligence about the application's behavior and traffic patterns. They can restart Apache to cause temporary service disruption. They cannot read authentication logs, cannot modify system configuration, cannot reach other users' workspaces, cannot escalate to root, and cannot affect any service beyond Apache. The blast radius is contained to the web application layer.

If auditor1's credentials are stolen:

The attacker gains read access to authentication logs, system logs, and Apache logs — intelligence about who accesses the system and when. They can read the sudo policy to understand privilege boundaries. They cannot write anywhere, cannot restart any service, cannot enter any user's home directory, and cannot escalate privileges in any way. The blast radius is limited to information disclosure — serious but contained and recoverable.

If ops1's credentials are stolen:

This is the highest-risk scenario. The attacker gains full sudo access and can do anything on the server. However, every action they take is logged by the audit daemon. The audit trail records every command, every file access, every privilege escalation with timestamps and session identifiers. The attacker cannot operate silently. Detection is certain — the only question is response time. This is why the operations role requires the strongest authentication controls and why auditd configuration in Chapter 8 is critical.

5.8 Summary

This chapter implemented the complete access control model on the backend server. Three roles were configured, each with precisely scoped permissions that reflect their operational responsibilities and nothing beyond them.

Developers received write access to the web deployment directory, read access to application logs, and a single sudo rule permitting only the restart of the Apache service. Every other elevated operation is denied. Their blast radius in the event of a credential compromise is limited to the web application layer.

Operations engineers received full sudo access to the entire server. This broad grant is justified by the infrastructure management responsibilities of the role and is controlled entirely through audit logging — every privileged command is recorded with full detail. The access is wide and the accountability is total.

Auditors received read-only access to system logs, application logs, and privilege configuration files. They cannot write to any file, cannot restart any service, and cannot enter any user's personal workspace. Their constraint is enforced at the filesystem level — not by policy alone but by the permission bits on every file and directory they interact with.

The cross-role isolation tests in section 5.7 validated that every boundary holds in both directions — permitted operations succeed and forbidden operations are denied. The permission model does not rely on trust. It relies on Linux kernel enforcement at the filesystem, service, and privilege layers simultaneously. Chapter 6 adds the final layer — network-level control through stateful packet inspection — completing the defense in depth architecture established in Chapter 2.

6 CHAPTER SIX: Network Security: Stateful Packet Filtering with nftables

6.1 Introduction

The previous two chapters established who can access the infrastructure and what they are permitted to do once inside. User accounts were created, group memberships were assigned, home directories were locked down, and sudo policies were written. Every permission decision was enforced at the filesystem and privilege level.

However, none of that protection matters if an attacker can reach services that should not be reachable at all. A correctly configured SSH daemon with key-based authentication is meaningless if an attacker can connect to an exposed database port. A strict sudo policy is irrelevant if a vulnerable web service is listening on a public interface with no network-level protection in front of it.

This chapter adds the network layer — the outermost boundary of the security model. Before any packet reaches sshd, before any authentication attempt is made, before any permission is checked, the firewall decides whether that packet has the right to exist on this machine at all. Packets that fail this check are dropped silently at the kernel level before any application process ever sees them.

On the bastion host, the firewall serves two distinct purposes simultaneously. First, it protects the bastion itself — allowing only SSH traffic inbound and dropping everything else. Second, it controls what passes through the bastion toward the internal network — forwarding only legitimate ProxyJump connections to the backend server while blocking all other forwarded traffic. These two responsibilities are handled by separate chains in the ruleset, each with its own policy and its own rules

6.2 Firewall Policy Definition

Before writing a single rule, the firewall policy must be defined in plain language. A ruleset written without a prior policy is a collection of decisions made in the moment — inconsistent, hard to audit, and impossible to maintain. The policy is the source of truth. Every rule that follows is a direct technical implementation of one line in this policy.

The bastion host has one network-facing responsibility: accept SSH connections from authorized engineers and forward them to the appropriate backend server. Everything else is noise. The policy reflects this:

Inbound traffic — what the bastion accepts for itself:

Traffic	Interface	Decision	Reason
All traffic	lo (loopback)	Accept	Internal process communication — system breaks without it
SSH port 22 — new connections	enp0s3	Accept	Engineers connecting from outside
Established and related packets	any	Accept	Ongoing conversations from approved connections
ICMP	any	Accept	Network diagnostics and connectivity testing
Everything else	any	Drop	Default — anything not explicitly permitted is rejected

Forwarded traffic — what the bastion passes through:

Traffic	From	To	Decision	Reason
Established and related	any	any	Accept	Return traffic for approved forwarded connections
Any traffic	enp0s9	enp0s3	Accept	Kali reaching internet for updates
SSH port 22 — new connections	enp0s3	enp0s9	Accept	ProxyJump — engineer tunneling to Kali
Everything else	any	any	Drop	Default — no unauthorized forwarding

Outbound traffic — what the bastion sends:

Traffic	Decision	Reason
All outbound	Accept	The bastion itself is trusted — unrestricted outbound

NAT — address translation:

Traffic	Action	Reason
Any traffic leaving enp0s3	Masquerade	Replace Kali's private IP with bastion's public IP so internet responses return correctly

This policy table is the single reference document for the entire ruleset. Every rule written in the following sections maps directly to one row in these tables. If a rule cannot be traced back to a row in this policy, it does not belong in the ruleset.

6.3 Understanding the Ruleset Structure

nftables organizes firewall rules in a three-level hierarchy: tables contain chains, and chains contain rules. Understanding this structure before reading or writing any rule is essential — without it, a ruleset is just a list of commands with no coherent mental model behind it.

6.3.1.1 Tables

A table is the top-level container. It defines the address family the rules apply to — inet covers both IPv4 and IPv6 simultaneously, ip covers IPv4 only. In this project two tables are used: inet BastionRules for all filtering decisions, and ip nat for address translation. Tables have no filtering logic themselves — they are organizational containers for chains.

6.3.1.2 Chains

A chain is where rules live. Each chain is attached to a specific hook point in the kernel's packet processing path — the moment in the packet's journey through the system where the chain's rules are evaluated. Three hook points are relevant to this project:

Chain type	Hook	When it runs
input	After routing decision — packet destined for this machine	Controls what reaches local processes
forward	After routing decision — packet passing through to another machine	Controls what the bastion forwards
output	Before routing — packet generated by this machine	Controls what the bastion sends

Each chain has a default policy — the verdict applied to any packet that reaches the end of the chain without matching any rule. In this project, input and forward chains use policy drop — anything not explicitly allowed is silently discarded. The output chain uses policy accept — the bastion's own outbound traffic is unrestricted.

6.3.1.3 Rules and Evaluation Order

Rules inside a chain are evaluated top to bottom. The first rule that matches a packet determines its verdict — no further rules are checked. This makes rule order critical:

The established,related rule appears before the SSH rule in the input chain. This is deliberate — the vast majority of packets on an active SSH connection are ESTABLISHED packets, not NEW ones. Placing the established,related rule first means most packets match immediately without being checked against every subsequent rule. Performance aside, order also affects correctness — a more specific rule placed before a general one can change the behavior of the entire chain in ways that are difficult to debug.

The default drop policy is the last line of defense. It is not a rule — it is the chain's answer to the question "what do I do with a packet that nobody claimed?" On a hardened bastion host, the answer is always: drop it.

6.3.1.4 Hook Points — Where Rules Actually Execute

When a packet arrives at the bastion's network interface, it does not go directly to sshd or any other application. It enters the kernel's networking stack and passes through a series of hook points where Netfilter — the kernel subsystem that nftables configures — evaluates it against the registered chains.

6.4 The Bastion Host Ruleset

With the policy defined and the structure understood, the ruleset can now be built. Every command in this section is a direct implementation of one decision from the policy table in section 6.2. Nothing is added beyond what the policy requires.

6.4.1 The BastionRules table

The first step is creating the table that will contain all filtering chains. The inet address family is chosen over ip because it handles both IPv4 and IPv6 traffic with a single ruleset — writing rules once that apply to both protocols rather than maintaining two separate tables.

Command:

```
nft add table inet BastionRules
```

The table name BastionRules is chosen deliberately over the generic filter name that most tutorials use. In a production environment with multiple tables serving different purposes, descriptive names make the ruleset self-documenting. An engineer reading inet BastionRules immediately understands what this table is for without needing additional context.

6.4.2 The input chain — protecting the bastion itself

The input chain protects the bastion itself. Every packet destined for a local process — sshd, systemd, any service running on the bastion — passes through this chain. The default policy is drop: anything not explicitly permitted is silently discarded before any application sees it.

Command:

```
nft add chain inet BastionRules input \ { type filter hook input priority filter \; policy drop \; }
```

The priority filter value places this chain at the standard filtering priority in the Netfilter hook system. Lower priority numbers execute earlier — filter is the conventional priority for security filtering chains.

Now we will explain each rule created in this chain and why we used it:

```
table inet BastionRules {
        chain input {
                type filter hook input priority filter; policy drop;
                iif "lo" accept
                ct state established,related accept
                tcp dport 22 ct state new accept
                ip protocol icmp accept
        }
```

Rule 1 — Loopback

Commad:

```
nft add rule inet BastionRules input iif "lo" accept
```

The loopback interface lo carries traffic between processes running on the same machine. systemd-resolved uses it for DNS. Applications use it to communicate with local services. Blocking loopback causes silent failures in system processes that are extremely difficult to diagnose. This rule accepts all loopback traffic unconditionally — it is the one rule in this chain that requires no threat analysis. No external packet can arrive on lo.

Rule 2 — Established and Related

Command:

```
nft add rule inet BastionRules input ct state established,related accept
```

This rule is placed second — before the SSH rule — because it matches the majority of packets on any active connection. Once an SSH session is established, every subsequent packet in that session carries the ESTABLISHED state. Without this rule those packets hit the DROP

policy and the connection dies immediately after the handshake. The connection tracking subsystem (ct) maintains a global state table — this rule consults that table and accepts any packet that belongs to a connection already approved by an earlier NEW packet.

Rule 3 — SSH

Command:

```
nft add rule inet BastionRules input tcp dport 22 ct state new accept
```

This is the core rule of the bastion host firewall. It accepts new TCP connections on port 22 — the only service the bastion exposes to the outside world. The ct state new constraint ensures this rule only matches the first packet of a new connection attempt, not arbitrary packets claiming to be SSH traffic. Combined with the ESTABLISHED rule above, this pair handles the complete lifecycle of an SSH connection: the NEW rule approves the initial handshake, the ESTABLISHED rule allows the ongoing conversation.

Rule 4 — ICMP

Command

```
nft add rule inet BastionRules input ip protocol icmp accept
```

ICMP is the protocol used by ping and network diagnostic tools. Allowing it enables connectivity testing from the internal network — verifying the bastion is reachable, measuring latency, and diagnosing network path issues during troubleshooting. In a production environment this rule would be restricted to specific source IP ranges. In this project it is permitted broadly for diagnostic convenience during development and testing.

6.4.3 The forward chain — controlling what passes through

The forward chain controls traffic that passes through the bastion rather than being destined for it. Two categories of traffic require forwarding: ProxyJump connections from engineers to the backend server, and outbound internet traffic from Kali for system updates.

Command:

```
nft add chain inet BastionRules forward \
{ type filter hook forward priority filter \; policy drop \; }
```

Now we will explain each rule in this chain and why we used it:

```
chain forward {
        type filter hook forward priority filter; policy drop;
        ct state established,related accept
        iif "enp0s9" oif "enp0s3" ct state established,related,new accept
        iif "enp0s3" oif "enp0s9" tcp dport 22 ct state established,related,new accept
}
```

Rule 1 — Allow Established and Related Forwarded Connections

Command:

```
nft add rule inet BastionRules forward ct state established,related accept
```

This rule serves a different purpose in the forward chain than it does in the input chain. When Kali sends a packet to the internet through the bastion, the connection tracking table records the full connection. When the internet server responds, that response packet arrives on enp0s3 heading toward enp0s9 — the reverse direction of the original request. No explicit forward rule covers this reverse direction. The established,related rule catches it: the kernel identifies the response as belonging to an already-approved connection and accepts it regardless of interface direction. Without this rule, Kali's outbound requests would leave the bastion but responses would never return — every apt update would hang indefinitely.

Rule 2 — Allow Outbound Forwarding from the Internal Network

Command:

```
nft add rule inet BastionRules forward \
  iif "enp0s9" oif "enp0s3" ct state new,established,related accept
```

This rule permits Kali to initiate connections to the internet through the bastion. Traffic arriving on the internal interface enp0s9 and leaving through the public interface enp0s3 is allowed for new connections and their ongoing traffic. This is the routing path for package updates, time synchronization, and any other outbound service Kali requires. The rule is intentionally broad on the destination — in a production environment outbound traffic would be restricted to specific package repository IP ranges. For this project, unrestricted outbound from Kali is acceptable.

Rule 3 — Allow Forwarding of SSH Traffic to Internal Hosts

Command:

```
nft add rule inet BastionRules forward \
  iif "enp0s3" oif "enp0s9" tcp dport 22 ct state new,established,related accept
```

This rule permits SSH traffic entering through the external interface to be forwarded to a host on the internal network. Unlike the input chain, which decides whether the bastion itself should receive a packet, the forward chain decides whether a packet may pass through the bastion toward another machine. By restricting the rule to TCP port 22, the bastion forwards only SSH traffic while refusing to act as a general-purpose router for other services.

Project usage: In Chapter 8, this rule is used to implement the SSH ProxyJump architecture, allowing administrators to securely reach Kali through the bastion without exposing Kali directly to the external network.

6.4.4 The output chain — bastion outbound traffic

The output chain controls traffic generated by the bastion itself — DNS queries, NTP synchronization, package downloads, SSH connections initiated from the bastion to internal servers. The policy is accept: the bastion's own outbound traffic is unrestricted.

Command:

```
nft add chain inet BastionRules output \
  { type filter hook output priority filter \; policy accept \; }
```

No explicit rules are needed in this chain. The accept policy handles all outbound traffic automatically. In a high-security production environment, the output chain would be restricted to specific destinations — preventing a compromised bastion from initiating arbitrary outbound connections. For this project, unrestricted output is acceptable and is documented as a known limitation of the simulation environment.

```
}

chain output {
        type filter hook output priority filter; policy accept;
}
```

6.4.5 The NAT table — masquerading Kali's traffic

The NAT table handles address translation for Kali's outbound internet traffic. Without this table, packets leaving the bastion on behalf of Kali carry Kali's private source IP address 192.168.10.10. Internet servers cannot route responses back to a private IP — the packets would be dropped at the first public router. The masquerade rule replaces Kali's private source IP with the bastion's public interface IP before the packet leaves, so responses return to the bastion which then forwards them back to Kali.

Command:

```
nft add table ip nat

nft add chain ip nat postrouting \ { type nat hook postrouting priority srcnat \; policy accept \; }

nft add rule ip nat postrouting oif "enp0s3" masquerade
```

The postrouting hook fires after the routing decision has been made — the packet knows where it is going and the NAT rule modifies its source address just before it leaves the interface. The masquerade action is a dynamic form of source NAT — it automatically uses whatever IP address is currently assigned to enp0s3, making the rule robust to IP address changes without requiring manual updates.

This is the same mechanism used by AWS NAT Gateway, home routers, and enterprise network address translation devices. The concept is identical at every scale — replace the private source address with a routable public address, track the translation in a table, and reverse it when the response arrives

```
        chain postrouting {
                type nat hook postrouting priority srcnat; policy accept;
                oif "enp0s3" masquerade
                oif "enp0s3" masquerade
        }
```

6.5 Applying and Persisting the Ruleset

The rules created in section 6.4 exist in the kernel's memory. They are active and enforcing right now — but they will not survive a reboot. When the system restarts, the kernel starts fresh with no firewall rules. Everything built in section 6.4 disappears.

Two steps are required to make the ruleset permanent: saving the current rules to a file that nftables reads on startup, and enabling the nftables service so it loads that file automatically at boot.

6.5.1 Step 1 — Validate the ruleset before saving:

Before saving anything, confirm the ruleset is syntactically correct and logically complete. nftables provides a validation command that checks the entire ruleset without applying it:

```
        chain postrouting {
                type nat hook postrouting priority srcnat; policy accept;
                oif "enp0s3" masquerade
                oif "enp0s3" masquerade
        }
}
root@akhoulid-VirtualBox:~# nft list ruleset table inet BastionRules
Error: syntax error, unexpected table, expecting end of file or newline or semicolon
list ruleset table inet BastionRules
             ^^^^^
root@akhoulid-VirtualBox:~# nft list table inet BastionRules
table inet BastionRules {
        chain input {
                type filter hook input priority filter; policy drop;
                iif "lo" accept
                ct state established,related accept
                tcp dport 22 ct state new accept
                ip protocol icmp accept
        }

        chain forward {
                type filter hook forward priority filter; policy drop;
                ct state established,related accept
                iif "enp0s9" oif "enp0s3" ct state established,related,new accept
                iif "enp0s3" oif "enp0s9" tcp dport 22 ct state established,related,new accept
        }

        chain output {
                type filter hook output priority filter; policy accept;
        }
}
root@akhoulid-VirtualBox:~#
```

6.5.2 Save the ruleset

The nftables service reads its rules from /etc/nftables.conf at startup. Save the current in-memory ruleset to this file:

```
root@akhoulid-VirtualBox:~# nft list ruleset > /etc/nftables.conf
# Warning: table ip filter is managed by iptables-nft, do not touch!
# Warning: table ip nat is managed by iptables-nft, do not touch!
# Warning: table ip6 nat is managed by iptables-nft, do not touch!
```

Verify the file was written correctly:

Command :

cat /etc/nftables.conf

```
root@akhoulid-VirtualBox:/etc# cd ../ && cd etc && cat nftables.conf | tail -n 20
table inet BastionRules {
        chain input {
                type filter hook input priority filter; policy drop;
                iif "lo" accept
                ct state established,related accept
                tcp dport 22 ct state new accept
                ip protocol icmp accept
        }

        chain forward {
                type filter hook forward priority filter; policy drop;
                ct state established,related accept
                iif "enp0s9" oif "enp0s3" ct state established,related,new accept
                iif "enp0s3" oif "enp0s9" tcp dport 22 ct state established,related,new accept
        }

        chain output {
                type filter hook output priority filter; policy accept;
        }
}
root@akhoulid-VirtualBox:/etc#
```

The file format is identical to what nft list ruleset displays — nftables reads and writes the same syntax. This makes the saved file human-readable and directly editable. An administrator can modify /etc/nftables.conf in a text editor and reload the service to apply changes, rather than running individual nft add rule commands.

6.5.3 Step 3 — Enable and start the nftables service

The nftables systemd service loads /etc/nftables.conf at boot and applies the ruleset automatically. Enable it so it starts on every reboot:

```
root@akhoulid-VirtualBox:/etc# systemctl enable nftables
systemctl start nftables
systemctl status nftables
Created symlink '/etc/systemd/system/sysinit.target.wants/nftables.service' → '/usr/lib/systemd/sy
● nftables.service - nftables
     Loaded: loaded (/usr/lib/systemd/system/nftables.service; enabled; preset: enabled)
     Active: active (exited) since Wed 2026-07-22 23:49:00 +01; 85ms ago
 Invocation: d459abc3c0fb4634899bb671ea52d8e0
       Docs: man:nft(8)
             http://wiki.nftables.org
    Process: 45313 ExecStart=/usr/sbin/nft -f /etc/nftables.conf (code=exited, status=0/SUCCESS)
   Main PID: 45313 (code=exited, status=0/SUCCESS)
   Mem peak: 7.3M
        CPU: 81ms

Jul 22 23:48:59 akhoulid-VirtualBox systemd[1]: Starting nftables.service - nftables...
Jul 22 23:49:00 akhoulid-VirtualBox systemd[1]: Finished nftables.service - nftables.
root@akhoulid-VirtualBox:/etc#
```

6.5.4 Step 4 — Test persistence after reboot

The only reliable way to confirm persistence is to reboot and verify the rules are still active:

```
root@akhoulid-VirtualBox:/etc# reboot

Broadcast message from root@akhoulid-VirtualBox on pts/1 (Wed 2026-07-22 23:49:44 +01):

The system will reboot now!

root@akhoulid-VirtualBox:/etc#
```

After the system comes back up, verify the ruleset loaded correctly:

```
table inet BastionRules {
        chain input {
                type filter hook input priority filter; policy drop;
                iif "lo" accept
                ct state established,related accept
                tcp dport 22 ct state new accept
                ip protocol icmp accept
        }

        chain forward {
                type filter hook forward priority filter; policy drop;
                ct state established,related accept
                iif "enp0s9" oif "enp0s3" ct state established,related,new accept
                iif "enp0s3" oif "enp0s9" tcp dport 22 ct state established,related,new accept
        }

        chain output {
                type filter hook output priority filter; policy accept;
        }
}
root@akhoulid-VirtualBox:/etc#
```

The rules are identical to those saved before the reboot. The nftables service loaded /etc/nftables.conf automatically during the boot sequence, before any network interface came up and before sshd started accepting connections. The firewall was active before the first packet could arrive.

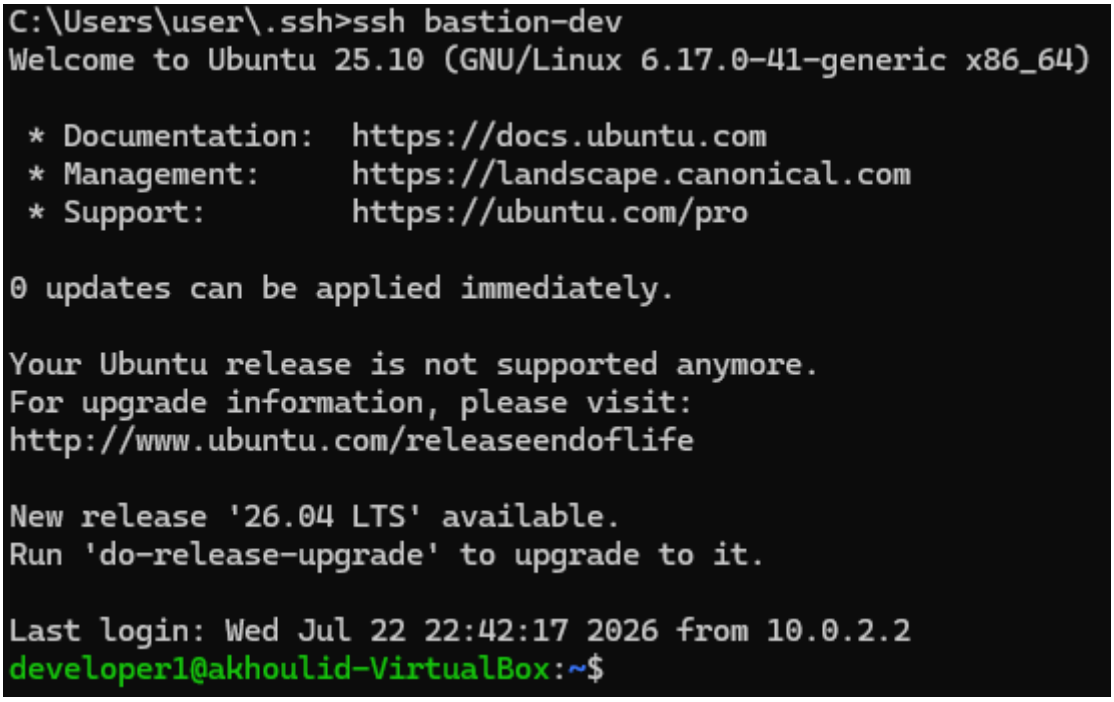
6.6 Validation and Testing

A firewall that has not been tested is not a firewall — it is a hope. Every rule written in section 6.4 must be verified in both directions: permitted traffic must pass, and blocked traffic must be dropped. Testing only the allowed paths leaves the blocked paths unverified. An incorrectly written rule might allow traffic it was designed to block, and without explicit testing that gap remains invisible.

This section validates every rule in the ruleset through direct testing. Each test has a clear expected result — pass or fail — and the actual result is documented with a screenshot

6.6.1 Test SSH access works through firewall

The most critical test — the firewall must not break the bastion's primary function. SSH access from Windows to the bastion must work exactly as it did before the firewall was applied.



The successful SSH connection confirms three rules are working simultaneously: the NEW rule accepted the initial SYN packet on port 22, the ESTABLISHED rule allowed all subsequent conversation packets, and the output chain's accept policy allowed the bastion's responses back to the client. If any of these three had failed, the connection would have been refused or dropped mid-handshake.

6.6.2 Test blocked ports are actually blocked (nmap)

Allowing SSH is only half the test. The other half is confirming that everything else is blocked. For this test, nmap is used from Kali to scan the bastion's public interface and confirm that no ports beyond 22 are reachable.

```
┌──(root㉿kali)-[/home]
└─# nmap -sS -p 1-1000 192.168.10.1
Starting Nmap 7.99 ( https://nmap.org ) at 2026-07-22 19:12 -0400
Nmap scan report for 192.168.10.1
Host is up (0.0024s latency).
Not shown: 999 filtered tcp ports (no-response)
PORT   STATE SERVICE
22/tcp open  ssh
MAC Address: 08:00:27:F3:C7:3C (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 5.80 seconds
```

The scan result shows only port 22 as open. Every other port in the range returns filtered — meaning the packets were sent but no response was received. This is the correct behavior of a DROP policy: packets are discarded silently without sending a TCP RST or ICMP unreachable message back to the scanner. An attacker scanning the bastion learns only that port 22 exists. The absence of responses on all other ports reveals nothing about what services might be running behind the firewall.

6.6.3 Test Kali can reach internet through bastion

This test validates both the forward chain's Kali-to-internet rule and the NAT masquerade rule. If either is broken, Kali cannot reach external package repositories

```
┌──(root㉿kali)-[/home]
└─# ping google.com
PING google.com (172.217.20.238) 56(84) bytes of data.
64 bytes from muc12s03-in-f14.1e100.net (172.217.20.238): icmp_seq=1 ttl=254 time
64 bytes from muc12s03-in-f14.1e100.net (172.217.20.238): icmp_seq=2 ttl=254 time
^C
── google.com ping statistics ──
2 packets transmitted, 2 received, 0% packet loss, time 1002ms
rtt min/avg/max/mdev = 24.989/25.195/25.401/0.206 ms

┌──(root㉿kali)-[/home]
└─# ping 8.8.8.8
PING 8.8.8.8 (8.8.8.8) 56(84) bytes of data.
64 bytes from 8.8.8.8: icmp_seq=1 ttl=254 time=30.6 ms
64 bytes from 8.8.8.8: icmp_seq=2 ttl=254 time=29.2 ms
^C
── 8.8.8.8 ping statistics ──
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 29.184/29.913/30.643/0.729 ms

┌──(root㉿kali)-[/home]
└─# apt update
Hit:1 https://packages.wazuh.com/4.x/apt stable InRelease
Get:2 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Packages [21.4 MB]
28% [3 Packages 11.1 MB/21.4 MB 52%]
```

Three separate confirmations at three different layers. The ping to 8.8.8.8 confirms IP-level connectivity — packets from Kali reach the internet and responses return. The ping to google.com confirms DNS resolution is working — Kali can resolve domain names through the bastion's routing path. The apt update confirms TCP connectivity to package repositories on port 443 — the full chain from Kali through the bastion's forward rules, through the NAT masquerade, to the internet, and back is functioning correctly.

6.6.4 Test ProxyJump works through firewall

The firewall has been configured to permit forwarding of SSH packets from enp0s3 to enp0s9 on TCP port 22. This rule provides the network path required by the ProxyJump architecture. The end-to-end functionality of ProxyJump is validated in Chapter X after the SSH client configuration has been introduced

6.6.5 Log dropped packets to confirm firewall is working

7 Chapter Seven — Intrusion Detection and Automated Response with fail2ban

7.1 Introduction

The previous chapters built a layered defense model for the bastion host. nftables controls which packets are allowed to reach the SSH daemon at the network level. SSH itself enforces cryptographic authentication and group-based access control. The permission model limits what authenticated users can do on the system.

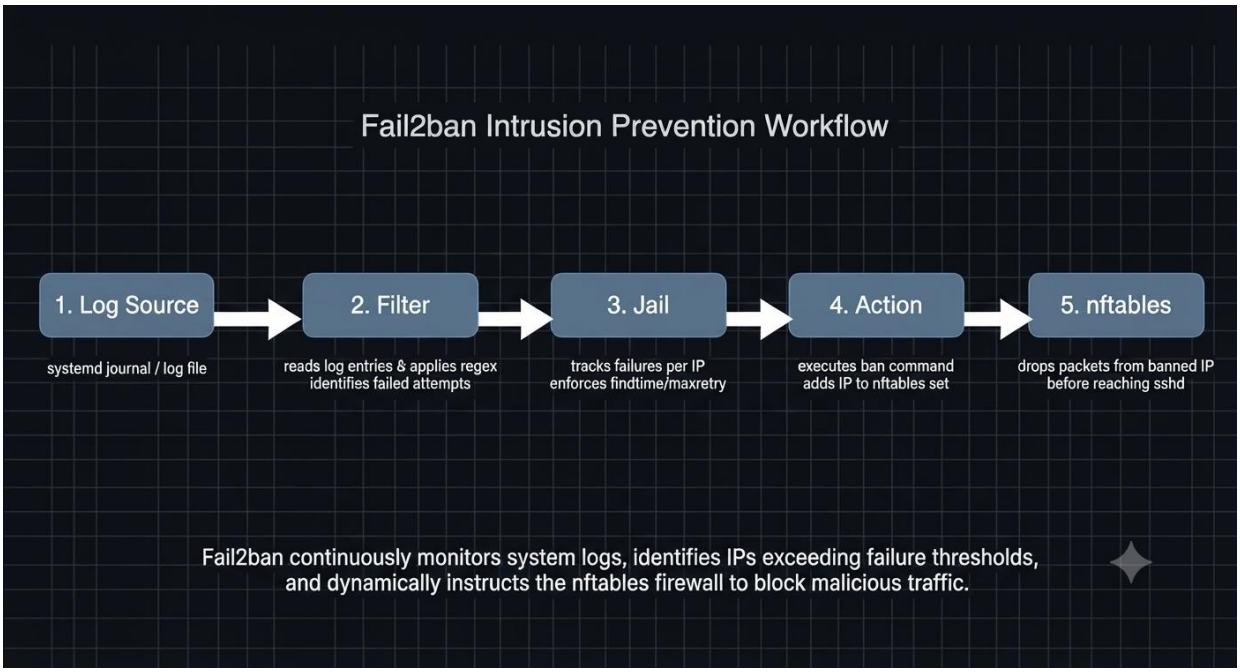
These layers are all passive — they respond to connection attempts but do not adapt to them. A determined attacker can attempt thousands of failed authentications against port 22, and as long as each individual attempt respects the configured limits, the static ruleset will never react. The attacker is slowed down but not stopped.

fail2ban closes this gap. It is an active defense layer that monitors authentication logs in real time, detects repeated failure patterns, and automatically adds firewall rules to ban the offending IP address for a configurable period. What was a slow, persistent brute force attempt becomes a blocked address after the fifth failure. The attack stops at the network layer before SSH ever sees another packet from that source.

This chapter is the final security layer of the bastion host infrastructure. It sits above nftables and below SSH in the defense stack — it reads what SSH reports and responds by updating what nftables enforces.

7.2 What fail2ban Is and How It Works

fail2ban is an intrusion prevention daemon written in Python. It has been actively maintained for over 20 years and ships with more than 100 pre-built filter definitions covering SSH, Apache, nginx, Postfix, and dozens of other services. Its architecture is built around three core concepts: filters, jails, and actions.



7.2.1 The three components explained

Filter — a regular expression definition that tells fail2ban what a failure looks like in the logs. Each filter targets one specific log message pattern. fail2ban ships with pre-built filters for most common services in /etc/fail2ban/filter.d/. When the log format does not match any built-in filter, a custom one must be written.

Jail — the configuration that ties everything together. A jail specifies which filter to use, which log source to monitor, how many failures constitute a ban trigger (maxretry), over what time window (findtime), and for how long to ban (bantime). Multiple jails can run simultaneously, each protecting a different service.

Action — what fail2ban executes when a jail triggers. Actions are scripts that add and remove firewall rules. In this project the action integrates directly with nftables — when an IP is banned, fail2ban adds it to a dedicated nftables set and all packets from that IP are rejected at the kernel level before SSH sees them. When the ban expires, fail2ban removes the IP from the set automatically.

7.3 Why fail2ban and Not Something Else

Several tools exist for brute force protection on Linux — sshguard, denyhosts, crowdsec, and manual nftables rate limiting among them. fail2ban was chosen for this project for three specific reasons.

Maturity and filter coverage. fail2ban has been in active development since 2004. Its filter library covers virtually every service that runs on Linux. The community has refined these filters against real-world log formats across two decades of production deployments. No other tool has comparable breadth.

Native nftables integration. fail2ban ships with a dedicated nftables action that creates its own table and set, inserts ban rules dynamically, and cleans up automatically when bans expire. The integration is clean — fail2ban manages its own nftables table (f2b-table) separately from the BastionRules table, so the two rulesets never conflict.

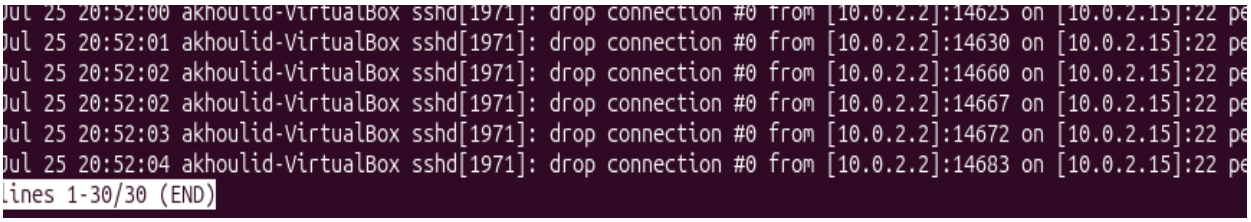
Jail flexibility. A single fail2ban instance can protect multiple services simultaneously with completely independent configurations. Adding Apache protection, SMTP protection, or any other service requires only a new jail file — no architectural changes.

Why fail2ban is the last layer. fail2ban depends on log entries generated by sshd. If the firewall had blocked the connection before sshd saw it, no log entry would exist and fail2ban would never know an attempt occurred. fail2ban therefore must sit above nftables in the stack — it reads what sshd reports and instructs nftables to act. The dependency order is: nftables filters first, sshd authenticates and logs, fail2ban reads those logs and updates nftables for future connections.

7.4 Why a Custom Filter Was Required

fail2ban ships with a built-in filter for SSH at /etc/fail2ban/filter.d/sshd.conf. This filter has been refined over many years against standard OpenSSH log formats. On most systems it works without modification.

Ubuntu 25.10 ships with OpenSSH 9.x which introduced a new connection penalty system. When a client fails authentication repeatedly, the daemon logs a message in a format that did not exist in older OpenSSH versions:



This message format is not covered by the built-in sshd.conf filter, which was written against older log patterns. Running fail2ban with the built-in filter against this system produced zero matches — the filter existed, the log entries existed, and nothing was detected.

A custom filter was written specifically for this message format. This is a normal part of working with fail2ban in production — the built-in filters cover the most common cases, but log formats change across service versions and distributions. When a built-in filter produces no matches, the diagnostic process is to examine the actual log entries, identify the exact message format, and write a regex that captures it.

The troubleshooting process that led to the custom filter is documented in section 7.6.

7.5 Configuration :

7.5.1 Installation

fail2ban is available in the Ubuntu package repositories and installs with all dependencies in a single operation. The service is enabled at installation time but requires explicit activation.

```
root@akhoulid-VirtualBox:~# apt install fail2ban
Installing:
  fail2ban

Installing dependencies:
  python3-jaraco.text  python3-pyasyncore  python3-pyinotify  python3-setuptools  python3-zipp  whois

Suggested packages:
  mailx  monit  sqlite3  python-pyinotify-doc  python-setuptools-doc

Summary:
  Upgrading: 0, Installing: 7, Removing: 0, Not Upgrading: 0
  Download size: 1192 kB
  Space needed: 6708 kB / 5573 MB available

Continue? [Y/n]
```

Once started, fail2ban runs as a background daemon. It reads its configuration, activates any enabled jails, and begins monitoring the configured log sources. At this point no custom jails are active — only the built-in defaults that ship with the package, none of which match the log format produced by OpenSSH 9.x on this system.

```
root@akhoulid-VirtualBox:/etc/fail2ban/filter.d# systemctl status fail2ban
● fail2ban.service - Fail2Ban Service
     Loaded: loaded (/usr/lib/systemd/system/fail2ban.service; enabled; preset: enabled)
     Active: active (running) since Sat 2026-07-25 20:51:20 +01; 42min ago
 Invocation: 49574b3fad294b5b90855c77fe3677b0
       Docs: man:fail2ban(1)
   Main PID: 24870 (fail2ban-server)
      Tasks: 7 (limit: 4148)
     Memory: 16.6M (peak: 21.1M)
        CPU: 23.133s
     CGroup: /system.slice/fail2ban.service
             └─24870 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

Jul 25 20:51:20 akhoulid-VirtualBox systemd[1]: Started fail2ban.service - Fail2Ban Service.
Jul 25 20:51:20 akhoulid-VirtualBox fail2ban-server[24870]: Server ready
```

7.5.2 Understanding the Configuration Structure

Before writing any configuration, it is important to understand how fail2ban organizes its files. The directory structure follows a deliberate convention that separates built-in configuration from local customization:

```
root@akhoulid-VirtualBox:/etc/fail2ban# ls
action.d        fail2ban.d  jail.conf  paths-arch.conf    paths-debian.conf
fail2ban.conf   filter.d    jail.d     paths-common.conf  paths-opensuse.conf
root@akhoulid-VirtualBox:/etc/fail2ban#
```

The correct practice is never to modify jail.conf or any built-in file directly. These files are overwritten on package upgrades. Local configuration goes into jail.d/ as .local files and into filter.d/ as separate files. This separation ensures that a package upgrade never destroys a working configuration.

This project requires two custom files: a filter that defines what a failed authentication looks like in the OpenSSH 9.x logs, and a jail that ties that filter to the log source and defines the ban policy.

7.5.3 The Custom Filter

fail2ban ships with a built-in SSH filter at /etc/fail2ban/filter.d/sshd.conf. This filter was written against older OpenSSH log formats and does not recognize the penalty messages produced by OpenSSH 9.x. Running the built-in filter against this system's journal produces zero matches. A custom filter is required.

The filter file contains a single failregex directive — a regular expression that identifies one failed authentication attempt. fail2ban applies this regex to every log entry it reads from the configured source. When the regex matches, fail2ban extracts the IP address captured by the <HOST> placeholder and increments the failure counter for that IP in the jail.

```
root@akhoulid-VirtualBox:/etc/fail2ban/filter.d# cat openssh-penalty.conf
[Definition]

failregex = ^.*drop connection .* from \[<HOST>\]:\d+ on .* penalty: failed authentication$

journalmatch = _SYSTEMD_UNIT=ssh.service + _COMM=sshd
root@akhoulid-VirtualBox:/etc/fail2ban/filter.d#
```

Three elements of this regex require explanation.

<HOST> is a fail2ban macro that expands to a pattern matching both IPv4 and IPv6 addresses. It is not a standard regex construct — fail2ban replaces it before compiling the expression. The captured address becomes the IP that fail2ban tracks and eventually bans.

The escaped brackets \[and \] are necessary because OpenSSH wraps the IP address in square brackets in the penalty message — from [10.0.2.2]:6107. The backslashes tell the regex engine to match literal bracket characters rather than treating them as grouping operators.

The ^.* at the beginning is the most important element and the one whose absence caused the initial failure. When fail2ban reads journal entries that were transported through syslog — which is the case for these OpenSSH penalty messages — it receives them with a timestamp and identifier prefix prepended to the actual message content. The log entry fail2ban evaluates is not the raw message but a prefixed version. The ^.* absorbs this prefix, allowing the rest of the pattern to match the message content regardless of what precedes it. This detail is documented in full in section 7.6.

7.5.4 The Jail Configuration

The jail is the central configuration unit in fail2ban. It ties the filter to a log source, defines the detection thresholds, and specifies what action to take when those thresholds are exceeded. The jail file is placed in /etc/fail2ban/jail.d/ with a .local extension, which tells fail2ban to treat it as a local override that takes precedence over built-in defaults.

```
root@akhoulid-VirtualBox:/etc/fail2ban/jail.d# cat openssh-penalty.local
[openssh-penalty]
enabled = true
port = ssh
filter = openssh-penalty
backend = systemd
journalmatch = _TRANSPORT=syslog _COMM=sshd
maxretry = 5
findtime = 1m
bantime = 1h
action = nftables[name=sshd, port=ssh, protocol=tcp]
root@akhoulid-VirtualBox:/etc/fail2ban/jail.d#
```

Each directive is a deliberate decision:

enabled = true activates this jail when fail2ban starts. Jails can be disabled without deleting their configuration by setting this to false — useful during maintenance or testing.

filter = openssh-penalty tells fail2ban to use the regex defined in openssh-penalty.conf. The name must match the filename exactly without the .conf extension.

backend = systemd instructs fail2ban to read from the systemd journal rather than a text file. This is the correct backend for Ubuntu systems using socket-activated SSH — the journal is the authoritative source for sshd messages on this system.

journalmatch = _SYSTEMD_UNIT=ssh.service + _COMM=sshd narrows which journal entries fail2ban reads. Without this directive, fail2ban would scan the entire journal — thousands of entries from every service on the system. This match restricts the scan to entries from the SSH service process only, which is both more efficient and more precise. The + operator requires both conditions to be true simultaneously.

maxretry = 5 and findtime = 1m define the detection threshold. Five failures within a sixty-second window trigger the ban. A legitimate engineer who misplaces their key will not reach this threshold. An automated brute force tool that fires hundreds of attempts per second will trigger it within the first second.

bantime = 1h sets the ban duration. After one hour fail2ban automatically removes the IP from the nftables set and the address can attempt connections again. In a production environment with persistent attackers, this value is typically increased to 24 hours or set to -1 for a permanent ban.

action = nftables[name=sshd, port=ssh, protocol=tcp] specifies the nftables action. When a ban triggers, fail2ban executes the nftables action script which creates a dedicated table (f2b-table), adds the banned IP to a set, and inserts a rule that rejects all packets from that IP on port 22. The ban operates at the kernel level — banned IPs never reach sshd at all.

7.5.5 Activation and Verification

With both files in place, fail2ban is restarted to load the new configuration:

```
root@akhoulid-VirtualBox:/etc/fail2ban/filter.d# fail2ban-client status openssh-penalty
Status for the jail: openssh-penalty
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     0
|  `- Journal matches:  _TRANSPORT=syslog _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     0
   `- Banned IP list:
```

The status output shows three important pieces of information. The filter section confirms the journal match is active and shows the current failure count — zero at this point because no attacks have occurred since the restart. The actions section shows zero current bans and an empty ban list. A jail that shows Currently failed: 0 and Currently banned: 0 immediately after a restart is in the correct initial state — it is active and monitoring, waiting for failures to accumulate.

7.6 Troubleshooting: Root Cause Analysis of Zero Filter Matches

After completing the configuration in section 7.5, the jail was active and the service was running without errors. However, fail2ban-client status openssh-penalty consistently reported zero failures despite repeated failed authentication attempts that were clearly visible in the system journal. The filter was syntactically correct, the jail was enabled, and the service was healthy — yet nothing was being detected.

This section documents the complete diagnostic process that identified and resolved the issue. The troubleshooting methodology is as important as the resolution itself — it demonstrates how to systematically isolate a fail2ban configuration problem when the obvious causes have already been ruled out.

7.6.1 Confirming the Log Entries Exist

The first question in any fail2ban debugging session is whether the log entries actually exist. A jail that detects nothing could mean the events are not being logged at all, or it could mean fail2ban cannot find them. These are different problems with different solutions.

```
nalty: failed authentication
root@akhoulid-VirtualBox:/etc/fail2ban/jail.d# journalctl | grep "penalty: failed" | tail -5
Jul 25 20:52:01 akhoulid-VirtualBox sshd[1971]: drop connection #0 from [10.0.2.2]:14630 on [10.0.2.15]:22 per
alty: failed authentication
Jul 25 20:52:02 akhoulid-VirtualBox sshd[1971]: drop connection #0 from [10.0.2.2]:14660 on [10.0.2.15]:22 per
alty: failed authentication
Jul 25 20:52:02 akhoulid-VirtualBox sshd[1971]: drop connection #0 from [10.0.2.2]:14667 on [10.0.2.15]:22 per
alty: failed authentication
Jul 25 20:52:03 akhoulid-VirtualBox sshd[1971]: drop connection #0 from [10.0.2.2]:14672 on [10.0.2.15]:22 per
alty: failed authentication
Jul 25 20:52:04 akhoulid-VirtualBox sshd[1971]: drop connection #0 from [10.0.2.2]:14683 on [10.0.2.15]:22 per
alty: failed authentication
root@akhoulid-VirtualBox:/etc/fail2ban/jail.d#
```

The entries were present. The problem was not absent logging — OpenSSH was generating the penalty messages correctly and they were reaching the journal. The issue was entirely on the fail2ban side.

7.6.2 Testing the Regex Directly

fail2ban provides a testing tool that applies a filter regex against a log source and reports how many lines matched. This isolates whether the problem is the regex itself or the journal match configuration:

```
root@akhoulid-VirtualBox:/etc/fail2ban/jail.d# fail2ban-regex systemd-journal /etc/fail2ban/filter.d/openssh-p
enalty.conf --print-all-matched

Running tests
=============

Use      filter file : openssh-penalty, basedir: /etc/fail2ban
Use         systemd journal
Use         encoding : UTF-8
Use    journal match : _SYSTEMD_UNIT=ssh.service + _COMM=sshd


Results
=======

Failregex: 0 total

Ignoreregex: 0 total

Lines: 142 lines, 0 ignored, 0 matched, 142 missed
[processed in 0.03 sec]

Missed line(s): too many to print.  Use --print-all-missed to print all 142 lines
root@akhoulid-VirtualBox:/etc/fail2ban/jail.d# fail2ban-regex 'systemd-journal[journalmatch=_TRANSPORT=syslog
_COMM=sshd]' /etc/fail2ban/filter.d/openssh-penalty.conf --print-all-matched
```

The result was zero matches across 112 journal lines. The regex was syntactically valid — fail2ban parsed it without errors — but it matched nothing. At this point the problem was narrowed to one of two causes: either the journal match was filtering out the relevant entries before the regex ever saw them, or the regex was correct in isolation but the actual string fail2ban was evaluating had a different format than expected.

7.6.3 Inspecting the Raw Journal Fields

To determine the exact format of the penalty messages as fail2ban sees them, the raw journal entry was inspected at the JSON level. The journal stores metadata fields alongside each message — these fields determine how fail2ban selects and reads entrie.:

We sued this command to find if there is a problem:

journalctl -o json | grep "penalty: failed" | head -1 | python3 -c "

import json, sys

```
data = json.loads(sys.stdin.read())

for k,v in sorted(data.items()): print(f'{k}: {v}')"
```

The output revealed the critical detail:

```
_SOURCE_REALTIME_TIMESTAMP: 1784950405245975
_SYSTEMD_CGROUP: /system.slice/ssh.service
_SYSTEMD_INVOCATION_ID: 7b58c8bc894e4b94ae54b25c95cf6736
_SYSTEMD_SLICE: system.slice
_SYSTEMD_UNIT: ssh.service
_TRANSPORT: syslog
_UID: 0
```

The _TRANSPORT: syslog field was the key. This field indicates that the journal entry arrived through the syslog pathway rather than through the native journal protocol. This distinction matters because fail2ban handles syslog-transported entries differently from natively journaled ones.

When fail2ban reads a journal entry with _TRANSPORT=syslog, it does not evaluate the raw MESSAGE field directly. Instead it reconstructs the full syslog line — including the timestamp, hostname, and process identifier — and evaluates that reconstructed string against the regex. The string fail2ban actually tested was not:

drop connection #0 from [10.0.2.2]:6107 on [10.0.2.15]:22 penalty: failed authentication

But rather the full syslog-prefixed version:

Jul 25 04:33:25 akhoulid-VirtualBox sshd[63032]: drop connection #0 from [10.0.2.2]:6107 on [10.0.2.15]:22 penalty: failed authentication

The original regex began with ^drop — anchoring the match to expect the string to start with the word drop. With the syslog prefix prepended, the string starts with a timestamp. The anchor never matched. Every penalty message was evaluated and rejected before the meaningful part of the regex could be compared against the message content.

This is a subtle but common class of fail2ban problem. The regex looks correct when tested against the raw message in isolation. It fails silently when applied through fail2ban's journal reader because the transport layer changes the string format. The only way to discover this is to inspect the raw journal fields and understand how fail2ban reconstructs the evaluated string from them.

7.6.4 The Fix

The solution was a single character change to the regex — replacing the strict start anchor ^drop with ^.*drop to absorb the syslog prefix:

Before:

```
[Definition]

failregex = ^drop connection .* from \[<HOST>\]:\d+ on .* penalty: failed authentication$

journalmatch = _SYSTEMD_UNIT=ssh.service + _COMM=sshd
```

After:

```
root@akhoulid-VirtualBox:/etc/fail2ban/filter.d# cat openssh-penalty.conf
[Definition]

failregex = ^.*drop connection .* from \[<HOST>\]:\d+ on .* penalty: failed authentication$

journalmatch = _SYSTEMD_UNIT=ssh.service + _COMM=sshd
root@akhoulid-VirtualBox:/etc/fail2ban/filter.d#
```

The .* matches any number of any characters — in practice it absorbs the timestamp, hostname, and process identifier that the syslog transport prepends. The rest of the regex then matches against the actual message content as expected.

After restarting fail2ban with the corrected filter, the jail immediately began detecting failures. The fix was confirmed by triggering several failed authentication attempts and observing the failure counter increment in real time.

7.6.5 What This Troubleshooting Reveals

This debugging session exposed a category of problem that appears frequently in production fail2ban deployments: the regex is correct, the jail is configured correctly, but the filter matches nothing because the string fail2ban evaluates has a different format than the raw log message suggests.

The diagnostic technique that resolved it — inspecting raw journal JSON fields to identify the transport mechanism — is the correct approach whenever a fail2ban filter produces no

matches despite confirmed log entries. The _TRANSPORT field determines how fail2ban reconstructs the evaluated string, and mismatches between expected and actual format are invisible without this inspection.

This problem occurs specifically with Ubuntu systems running OpenSSH with systemd socket activation. The penalty messages from the main sshd listener process use syslog transport rather than the native journal protocol, which is not the case for session-level messages like successful logins. A filter written against session messages works correctly. A filter written against penalty messages requires the ^.* prefix to handle the transport difference.

7.7 Validation and Testing

A firewall rule that has never been tested is an assumption. A fail2ban jail that has never triggered is an untested claim. This section validates the complete detection and response chain — from the failed authentication attempt through to the nftables ban rule — by deliberately triggering the jail and observing each step.

From Windows, connect repeatedly with an incorrect key to generate authentication failures:

```
C:\Users\user\.ssh>ssh bastion-dev
developer1@127.0.0.1: Permission denied (publickey).

C:\Users\user\.ssh>ssh bastion-dev
developer1@127.0.0.1: Permission denied (publickey).

C:\Users\user\.ssh>ssh bastion-dev
developer1@127.0.0.1: Permission denied (publickey).

C:\Users\user\.ssh>ssh bastion-dev
Connection closed by 127.0.0.1 port 2000

C:\Users\user\.ssh>ssh bastion-dev
Connection closed by 127.0.0.1 port 2000

C:\Users\user\.ssh>ssh bastion-dev
Connection closed by 127.0.0.1 port 2000
```

On the bastion, watch the jail status update in real time:

```
Status for the jail: openssh-penalty
|- Filter
|  |- Currently failed: 1
|  |- Total failed:     6
|  `- Journal matches:  _TRANSPORT=syslog _COMM=sshd
`- Actions
   |- Currently banned: 1
   |- Total banned:     1
   `- Banned IP list:   10.0.2.2
root@akhoulid-VirtualBox:/etc/fail2ban/filter.d# nft list ruleset
```

The status confirms the complete detection chain worked. fail2ban read the penalty messages from the journal, the regex matched them, the failure counter reached the maxretry threshold of 5 within the findtime window of 1 minute, and the nftables action executed to ban the source IP.

7.7.1 Confirming the nftables Rule

When fail2ban bans an IP it does not modify the BastionRules table configured in Chapter 6. It creates its own independent nftables table and manages it autonomously. Inspect it:

```
table inet f2b-table {
        set addr-set-sshd {
                type ipv4_addr
                elements = { 10.0.2.2 }
        }

        chain f2b-chain {
                type filter hook input priority filter - 1; policy accept;
                tcp dport 22 ip saddr @addr-set-sshd reject with icmp port-unreachable
        }
}
```

The output shows fail2ban's independent table structure. It contains a chain named f2b-chain hooked into the input path at priority -1 — which means it executes before the BastionRules input chain. A banned IP is rejected before it even reaches the rules in Chapter 6. The set named addr-set-sshd holds the banned addresses. The rule rejects all TCP packets on port 22 from any address in that set.

This architecture is intentional. fail2ban manages its own table so that its dynamic bans never interfere with the static ruleset. The BastionRules table remains stable and predictable. The f2b-table changes dynamically as bans are added and expired.

7.7.2 Confirming the Banned IP Cannot Connect

The most direct test: attempt to connect from the banned IP and confirm the connection is refused at the network level before sshd responds:

```
C:\Users\user\.ssh>ssh bastion-dev

C:\Users\user\.ssh>ssh bastion-dev
```

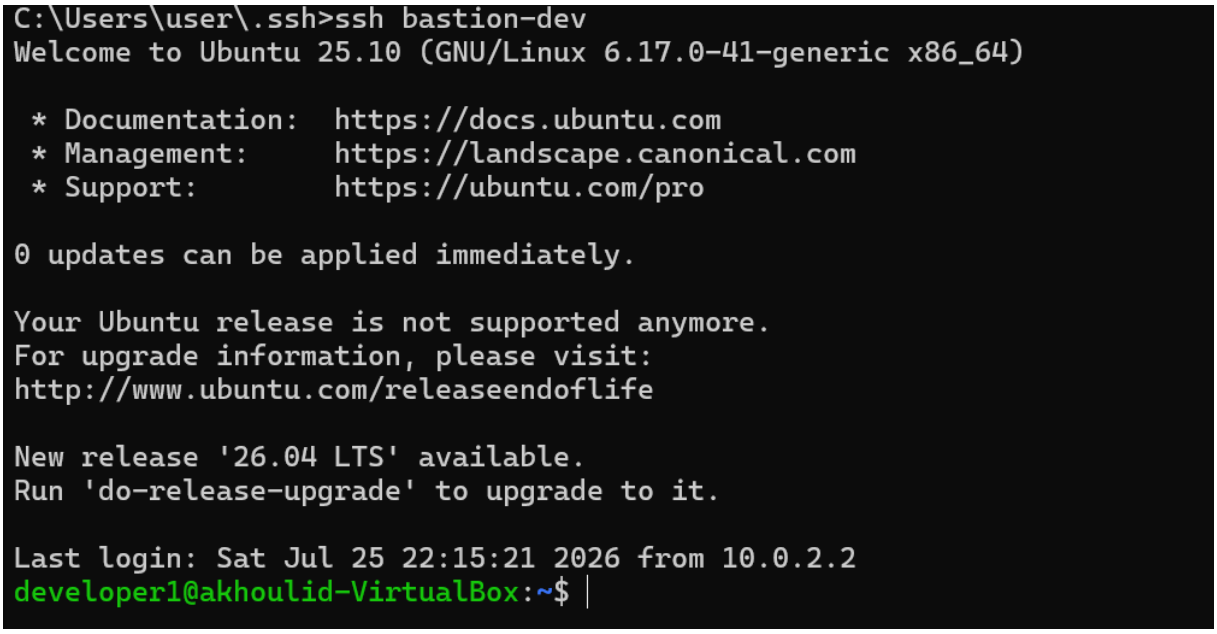
The connection does not reach sshd. The nftables rule rejects the packet before the SSH daemon processes it. From the client's perspective the connection is refused immediately — there is no SSH banner, no authentication prompt, no error message from the application layer. The rejection happens at the kernel level and is indistinguishable from the host being offline.

7.7.3 Manual Unban and Recovery

fail2ban automatically removes bans after the bantime expires. For testing purposes bans can be removed manually:

```
root@akhoulid-VirtualBox:/etc/fail2ban# fail2ban-client unban 10.0.2.2
fail2ban-client status openssh-penalty
1
Status for the jail: openssh-penalty
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     5
|  `- Journal matches:  _TRANSPORT=syslog _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     2
   `- Banned IP list:
```

After unbanning, SSH access from the previously banned IP works correctly:



This confirms the complete lifecycle: detection, ban, enforcement, and recovery all function correctly. The jail is operational.

7.8 Summary

fail2ban completes the active defense layer of the bastion host. Where nftables is a static barrier and SSH is a cryptographic checkpoint, fail2ban is a dynamic sensor that adapts the firewall in response to observed behavior. An attacker who triggers five authentication failures within sixty seconds is banned at the network level for one hour — automatically, without administrator intervention, before any further attempts can be made.

The configuration required a custom filter because OpenSSH 9.x introduced a penalty log format that the built-in filters do not recognize. The troubleshooting process that produced the working filter — inspecting raw journal fields, identifying the syslog transport mechanism, and correcting the regex anchor — is documented in section 7.6 as a reusable diagnostic methodology for this class of fail2ban problem.

The complete security model of the bastion host now operates across four independent layers:

Layer	Mechanism	What it prevents
Network filtering	nftables BastionRules	Unauthorized ports · invalid packets · unauthorized forwarding
Active intrusion detection	fail2ban + f2b-table	Brute force · automated scanners · repeated failures
Cryptographic authentication	OpenSSH	Password attacks · unauthorized keys · non-allowed groups
Access control	Linux RBAC + sudo policy	Privilege escalation · lateral movement · unauthorized file access

Each layer is independent. Each assumes the previous one has already been compromised. Together they implement the Zero Trust principle established in Chapter 2: never trust, always verify, assume breach, and contain the blast radius at every layer.